John Harrison

Handbook of Practical Logic and Automated Reasoning

CAMBRIDGE

CAMBRIDGE www.cambridge.org/9780521899574

This page intentionally left blank

HANDBOOK OF PRACTICAL LOGIC AND AUTOMATED REASONING

John Harrison

The sheer complexity of computer systems has meant that automated reasoning, i.e. the use of computers to perform logical inference, has become a vital component of program construction and of programming language design. This book meets the demand for a self-contained and broad-based account of the concepts, the machinery and the use of automated reasoning. The mathematical logic foundations are described in conjunction with their practical application, all with the minimum of prerequisites.

The approach is constructive, concrete and algorithmic: a key feature is that methods are described with reference to actual implementations (for which code is supplied) that readers can use, modify and experiment with.

This book is ideally suited for those seeking a one-stop source for the general area of automated reasoning. It can be used as a reference, or as a place to learn the fundamentals, either in conjunction with advanced courses or for self study.

John Harrison is a Principal Engineer at Intel Corporation in Portland, Oregon. He specialises in formal verification, automated theorem proving, floating-point arithmetic and mathematical algorithms.

HANDBOOK OF PRACTICAL LOGIC AND AUTOMATED REASONING

JOHN HARRISON

CAMBRIDGE UNIVERSITY PRESS
Cambridge, New York, Melbourne, Madrid, Cape Town, Singapore, São Paulo

Cambridge University Press
The Edinburgh Building, Cambridge CB2 8RU, UK
Published in the United States of America by Cambridge University Press, New York

www.cambridge.org
Information on this title: www.cambridge.org/9780521899574

First published in print format 2009

ISBN-13 978-0-511-50156-2 eBook (Adobe Reader)

ISBN-13 978-0-521-89957-4 hardback

To Porosusha

When a man *Reasoneth*, hee does nothing else but conceive a summe totall, from *Addition* of parcels.

For as Arithmeticians teach to adde and substract in *numbers*; so the Geometricians teach the same in *lines*, *figures* (solid and superficiall,) *angles*, *proportions*, *times*, degrees of *swiftnesse*, *force*, *power*, and the like; The Logicians teach the same in *Consequences of words*; adding together *two Names*, to make an *Affirmation*; and *two Affirmations*, to make a *Syllogisme*; and many *Syllogismes* to make a *Demonstration*; and from the *summe*, or *Conclusion* of a *Syllogisme*, they substract one *Proposition*, to finde the other.

For REASON, in this sense, is nothing but *Reckoning* (that is, Adding and Substracting) of the Consequences of generall names agreed upon, for the *marking* and *signifying* of our thoughts.

And as in Arithmetique, unpractised men must, and Professors themselves may often erre, and cast up false; so also in any other subject of Reasoning, the ablest, most attentive, and most practised men, may deceive themselves and inferre false Conclusions; Not but that Reason it selfe is always Right Reason, as well as Arithmetique is a certain and infallible Art: But no one mans Reason, nor the Reason of any one number of men, makes the certaintie; no more than an account is therefore well cast up, because a great many men have unanimously approved it.

Thomas Hobbes (1588–1697), '*Leviathan, or The Matter, Forme, & Power of a Common-Wealth Ecclesiasticall and Civill*'. Printed for ANDREW CROOKE, at the Green Dragon in St. *Pauls* Church-yard, 1651.

Contents

Preface

This book is about computer programs that can perform *automated reasoning*. I interpret 'reasoning' quite narrowly: the emphasis is on formal deductive inference rather than, for example, poker playing or medical diagnosis. On the other hand I interpret 'automated' broadly, to include interactive arrangements where a human being and machine reason together, and I'm always conscious of the applications of deductive reasoning to real-world problems. Indeed, as well as being inherently fascinating, the subject is deriving increasing importance from its industrial applications.

This book is intended as a first introduction to the field, and also to logical reasoning itself. No previous knowledge of mathematical logic is assumed, although readers will inevitably find some prior experience of mathematics and of computer programming (especially in a functional language like OCaml, F#, Standard ML, Haskell or LISP) invaluable. In contrast to the many specialist texts on the subject, this book aims at a broad and balanced general introduction, and has two special characteristics.

- Pure logic and automated theorem proving are explained in a closely intertwined manner. Results in logic are developed with an eye to their role in automated theorem proving, and wherever possible are developed in an explicitly computational way.
- Automated theorem proving methods are explained with reference to actual concrete implementations, which readers can experiment with if they have convenient access to a computer. All code is written in the high-level functional language OCaml.

Although this organization is open to question, I adopted it after careful consideration, and extensive experimentation with alternatives. A more detailed self-justification follows, but most readers will want to skip straight to the main content, starting with 'How to read this book' on page xvi.

Ideological orientation

This section explains in more detail the philosophy behind the present text, and attempts to justify it. I also describe the focus of this book and major topics that I do not include. To fully appreciate some points made in the discussion, knowledge of the subject matter is needed. Readers may prefer to skip or skim this material.

My primary aim has been to present a broad and balanced discussion of many of the principal results in automated theorem proving. Moreover, readers mainly interested in pure mathematical logic should find that this book covers most of the traditional results found in mainstream elementary texts on mathematical logic: compactness, Löwenheim–Skolem, completeness of proof systems, interpolation, Gödel's theorems etc. But I consistently strive, even when it is not directly necessary as part of the code of an automated prover, to present results in a concrete, explicit and algorithmic fashion, usually involving real code that can actually be experimented with and used, at least in principle. For example:

- the proof of the interpolation theorem in Section 5.13 contains an algorithm for constructing interpolants, utilizing earlier theorem proving code;
- decidability based on the finite model property is demonstrated in Section 5.5 by explicitly interleaving proving and refuting code rather than a general appeal to Theorem 7.13.

I hope that many readers will share my liking for this concrete hands-on style. Formal logic usually involves a considerable degree of care over tedious syntactic details. This can be quite painful for the beginner, so teachers and authors often have to make the unpalatable choice between (i) spelling everything out in excruciating detail and (ii) waving their hands profusely to cover over sloppy explanations. While teachers rightly tend to recoil from (i), my experience of teaching has shown me that many students nevertheless resent the feeling of never being told the whole story. By implementing things on a computer, I think we get the best of both worlds: the details are there in precise formal detail, but we can mostly let the computer worry about their unpleasant consequences.

It is true that mathematics in the last 150 years has become more abstractly set-theoretic and less constructive. This is particularly so in contemporary model theory, where traditional topics that lie at the historical root of the subject are being de-emphasized. But I'm not alone in swimming against this tide, for the rise of the computer is helping to restore the place of explicit algorithmic methods in several areas of mathematics. This is

particularly notable in algebraic geometry and related areas (Cox, Little and O'Shea 1992; Schenk 2003) where computer algebra and specifically Gröbner bases (see Section 5.11) have made considerable impact. But similar ideas are being explored in other areas, even in category theory (Rydeheard and Burstall 1988), often seen as the quintessence of abstract nonconstructive mathematics. I can do no better than quote Knuth (1974) on the merits of a concretely algorithmic point of view in mathematics generally:

> For three years I taught a sophomore course in abstract algebra for mathematics majors at Caltech, and the most difficult topic was always the study of "Jordan canonical forms" for matrices. The third year I tried a new approach, by looking at the subject algorithmically, and suddenly it became quite clear. The same thing happened with the discussion of finite groups defined by generators and relations, and in another course with the reduction theory of binary quadratic forms. By presenting the subject in terms of algorithms, the purpose and meaning of the mathematical theorems became transparent.
>
> Later, while writing a book on computer arithmetic [Knuth (1969)], I found that virtually every theorem in elementary number theory arises in a natural, motivated way in connection with the problem of making computers do high-speed numerical calculations. Therefore I believe that the traditional courses in number theory might well be changed to adopt this point of view, adding a practical motivation to the already beautiful theory.

In the case of logic, this approach seems especially natural. From the very earliest days, the development of logic was motivated by the desire to reduce reasoning to calculation: the word *logos*, the root of 'logic', can mean not just logical thought but also computation or 'reckoning'. More recently, it was decidability questions in logic that led Turing and others to define precisely the notion of a 'computable function' and set up the abstract models that delimit the range of algorithmic methods. This relationship between logic and computation, which dates from before the Middle Ages, has continued to the present day. For example, problems in the design and verification of computer systems are stimulating more research in logic, while logical principles are playing an increasingly important role in the design of programming languages. Thus, logical reasoning can be seen not only as one of the many beneficiaries of the modern computer age, but as its most important intellectual wellspring.

Another feature of the present text that some readers may find surprising is its systematically model-theoretic emphasis; by contrast many other texts such as Goubault-Larrecq and Mackie (1997) place proof theory at the centre. I introduce traditional proof systems late (Chapter 6), and I hardly mention, and never exploit, structural properties of natural deduction or sequent calculus proofs. While these topics are fascinating, I believe that all the traditional computer-based proof methods for classical logic can be presented

perfectly well without them. Indeed the special refutation-complete calculi for automated theorem proving (binary resolution, hyperresolution, etc.) also provide strong results on canonical forms for proofs. In some situations these are even more convenient for theoretical results than results from Gentzen-style proof theory (Matiyasevich 1975), as with our proof of the Nullstellensatz in Section 5.10 à la Lifschitz (1980). In any case, the details of particular proof systems can be much less significant for automated reasoning than the way in which the corresponding search space is examined. Note, for example, how different tableaux and the inverse method are, even though they can both be understood as search for cut-free sequent proofs.

I wanted to give full, carefully explained code for all the methods described. (In my experience it's easy to underestimate the difficulty in passing from a straightforward-looking algorithm to a concrete implementation.) In order to present real executable code that's almost as readable as the kind of pseudocode often used to describe algorithms, it seemed necessary to use a very high-level language where concrete issues of data representation and memory allocation can be ignored. I selected the functional programming language Objective CAML (OCaml) for this purpose. OCaml is a descendant of Edinburgh ML, a programming language specifically designed for writing theorem provers, and several major systems are written in it.

A drawback of using OCaml (rather than say, C or Java) is that it will be unfamiliar to many readers. However, I only use a simple subset, which is briefly explained in Appendix 2; the code is functional in style with no assignments or sequencing (except for producing diagnostic output). In a few cases (e.g. threading the state through code for binary decision diagrams), imperative code might have been simpler, but it seemed worthwhile to stick to the simplest subset possible. Purely functional programming is particularly convenient for the kind of tinkering that I hope to encourage, since one doesn't have to worry about accidental side-effects of one computation on others.

I will close with a quotation from McCarthy (1963) that nicely encapsulates the philosophy underlying this text, implying as it does the potential new role of logic as a truly *applied* science.

> It is reasonable to hope that the relationship between computation and mathematical logic will be as fruitful in the next century as that between analysis and physics in the last.

What's not in this book

Although I aim to cover a broad range of topics, selectivity was essential to prevent the book from becoming unmanageably huge. I focus on theories in classical one-sorted first-order logic, since in this coherent setting many of

the central methods of automated reasoning can be displayed. Not without regret, I have therefore excluded from serious discussion major areas such as model checking, inductive theorem proving, many-sorted logic, modal logic, description logics, intuitionistic logic, lambda calculus, higher-order logic and type theory. I believe, however, that this book will prepare the reader quite well to proceed with any of those areas, many of which are best understood precisely in terms of their contrast with classical first-order logic.

Another guiding principle has been to present topics only when I felt competent to do so at a fairly elementary level, without undue technicalities or difficult theory. This has meant the neglect of, for example, ordered paramodulation, cylindrical algebraic decomposition and Gödel's second incompleteness theorem. However, in such cases I have tried to give ample references so that interested readers can go further on their own.

Acknowledgements

This book has taken many years to evolve in haphazard fashion into its current form. During this period, I worked in the University of Cambridge Computer Laboratory, Åbo Akademi University/TUCS and Intel Corporation, as well as spending shorter periods visiting other institutions; I'm grateful above all to Tania and Yestin, for accompanying me on these journeys and tolerating the inordinate time I spent working on this project. It would be impossible to fairly describe here the extent to which my thinking has been shaped by the friends and colleagues that I have encountered over the years. But I owe particular thanks to Mike Gordon, who first gave me the opportunity to get involved in this fascinating field.

I wrote this book partly because I knew of no existing text that presents the range of topics in logic and automated reasoning that I wanted to cover. So the general style and approach is my own, and no existing text can be blamed for its malign influence. But on the purely logical side, I have mostly followed the presentation of basic metatheorems given by Kreisel and Krivine (1971). Their elegant development suits my purposes precisely, being purely model-theoretic and using the workaday tools of automated theorem proving such as Skolemization and the (so-called) Herbrand theorem. For example, the appealingly algorithmic proof of the interpolation theorem given in Section 5.13 is essentially theirs.

Though I have now been a researcher in automated reasoning for almost 20 years, I'm still routinely finding old results in the literature of which I was previously unaware, or learning of them through personal contact with

colleagues. In this connection, I'm grateful to Grigori Mints for pointing me at Lifschitz's proof of the Nullstellensatz (Section 5.10) using resolution proofs, to Loïc Pottier for telling me about Hörmander's algorithm for real quantifier elimination (Section 5.9), and to Lars Hörmander himself for answering my questions on the genesis of this procedure.

I've been very lucky to have numerous friends and colleagues comment on drafts of this book, offer welcome encouragement, take up and modify the associated code, and even teach from it. Their influence has often clarified my thinking and sometimes saved me from serious errors, but needless to say, they are not responsible for any remaining faults in the text. Heartfelt thanks to Rob Arthan, Jeremy Avigad, Clark Barrett, Robert Bauer, Bruno Buchberger, Amine Chaieb, Michael Champigny, Ed Clarke, Byron Cook, Nancy Day, Torkel Franzén (who, alas, did not live to see the finished book), Dan Friedman, Mike Gordon, Alexey Gotsman, Jim Grundy, Tom Hales, Tony Hoare, Peter Homeier, Joe Hurd, Robert Jones, Shuvendu Lahiri, Arthur van Leeuwen, Sean McLaughlin, Wojtek Moczydlowski, Magnus Myreen, Tobias Nipkow, Michael Norrish, John O'Leary, Cagdas Ozgenc, Heath Putnam, Tom Ridge, Konrad Slind, Jørgen Villadsen, Norbert Voelker, Ed Westbrook, Freek Wiedijk, Carl Witty, Burkhart Wolff, and no doubt many other correspondents whose contributions I have thoughtlessly forgotten about over the course of time, for their invaluable help.

Even in the age of the Web, access to good libraries has been vital. I want to thank the staff of the Cambridge University Library, the Computer Laboratory and DPMMS libraries, the mathematics and computer science libraries of Åbo Akademi, and more recently Portland State University Library and Intel Library, who have often helped me track down obscure references. I also want to acknowledge the peerless Powell's Bookstore (`www.powells.com`), which has proved to be a goldmine of classic logic and computer science texts.

Finally, let me thank Frances Nex for her extraordinarily painstaking copyediting, as well as Catherine Appleton, Charlotte Broom, Clare Dennison and David Tranah at Cambridge University Press, who have shepherded this book through to publication despite my delays, and have provided invaluable advice, backed up by the helpful comments of the Press's anonymous reviewers.

How to read this book

The text is designed to be read sequentially from beginning to end. However, after a study of Chapter 1 and a good part of each of Chapters 2 and 3, the reader may be in a position to dip into other parts according to taste.

To support this, I've tried to make some important cross-references explicit, and to avoid over-elaborate or non-standard notation where possible.

Each chapter ends with a number of exercises. These are almost never intended to be routine, and some are very difficult. This reflects my belief that it's more enjoyable and instructive to solve one really challenging problem than to plod through a large number of trivial drill exercises. The reader shouldn't be discouraged if most of them seem too hard. They are all optional, i.e. the text can be understood without doing any of them.

The mathematics used in this book

Mathematics plays a double role in this book: the subject matter itself is treated mathematically, and automated reasoning is also applied to some problems *in* mathematics. But for the most part, the mathematical knowledge needed is not all that advanced: basic algebra, sets and functions, induction, and perhaps most fundamentally, an understanding of the notion of a proof. In a few places, more sophisticated analysis and algebra are used, though I have tried to explain most things as I go along. Appendix 1 is a summary of relevant mathematical background that the reader might refer to as needed, or even skim through at the outset.

The software in this book

An important part of this book is the associated software, which includes simple implementations, in the OCaml programming language, of the various theorem-proving techniques described. Although the book can generally be understood without detailed study of the code, explanations are often organized around it, and code is used as a proxy for what would otherwise be a lengthy and formalistic description of a syntactic process. (For example, the completeness proof for first-order logic in Sections 6.4–6.8 and the proof of Σ_1-completeness of Robinson arithmetic in Section 7.6 are essentially detailed informal arguments that some specific OCaml functions always work.) So without at least a weak impressionistic idea of how the code works, you will probably find some parts of the book heavy going.

Since I expect that many readers will have little or no experience of programming, at least in a functional language like OCaml, I have summarized some of the key ideas in Appendix 2. I don't delude myself into believing that reading this short appendix will turn a novice into an accomplished functional programmer, but I hope it will at least provide some orientation, and it does include references that the reader can pursue if necessary. In fact,

the whole book can be considered an extended case study in functional programming, illustrating many important ideas such as structured data types, recursion, higher-order functions, continuations and abstract data types.

I hope that many readers will not only look at the code, but actually run it, apply it to new problems, and even try modifying or extending it. To do any of these, though, you will need an OCaml interpreter (see Appendix 2 again). The theorem-proving code itself is almost entirely listed in piecemeal fashion within the text. Since the reader will presumably profit little from actually typing it in, all the code can be downloaded from the website for this book (www.cambridge.org/9780521899574) and then just loaded into the OCaml interpreter with a few keystrokes or cut-and-pasted one phrase at a time.

In the future, I hope to make updates to the code and perhaps ports to other languages available at the same URL. More details can be found there about how to run the code, and hence follow along the explanations given in the book while trying out the code in parallel, but I'll just mention a couple of important points here. Probably the easiest way to proceed is to load the entire code associated with this book, e.g. by starting the OCaml interpreter `ocaml` in the directory (folder) containing the code and typing:

```
#use "init.ml";;
```

The default environment is set up to automatically parse anything in French-style ≪quotations≫ as a first-order formula. To use some code in Chapter 1 you will need to change this to parse arithmetic expressions:

```
let default_parser = make_parser parse_expression;;
```

and to use some code in Chapter 2 on propositional logic, you will need to change it to parse propositional formulas:

```
let default_parser = parse_prop_formula;;
```

Otherwise, you can more or less dip into any parts of the code that interest you. In a very few cases, a basic version of a function is defined first as part of the expository flow but later replaced by a more elaborate or efficient version with the same name. The default environment in such cases will always give you the latest one, and if you want to follow the exposition conscientiously you may want to cut-and-paste the earlier version from its source file.

The code is mainly intended to serve a pedagogical purpose, and I have always given clarity and/or brevity priority over efficiency. Still, it sometimes

might be genuinely useful for applications. In any case, before using it, please pay careful attention to the (minimal) legal restrictions listed on the website. Note also that Stålmarck's algorithm (Section 2.10) is patented, so the code in the file `stal.ml` should not be used for commercial applications.

1

Introduction

In this chapter we introduce logical reasoning and the idea of mechanizing it, touching briefly on important historical developments. We lay the groundwork for what follows by discussing some of the most fundamental ideas in logic as well as illustrating how symbolic methods can be implemented on a computer.

1.1 What is logical reasoning?

There are many reasons for believing that something is true. It may seem obvious or at least immediately plausible, we may have been told it by our parents, or it may be strikingly consistent with the outcome of relevant scientific experiments. Though often reliable, such methods of judgement are not infallible, having been used, respectively, to persuade people that the Earth is flat, that Santa Claus exists, and that atoms cannot be subdivided into smaller particles.

What distinguishes *logical* reasoning is that it attempts to avoid any unjustified assumptions and confine itself to inferences that are infallible and beyond reasonable dispute. To avoid making any unwarranted assumptions, logical reasoning cannot rely on any special properties of the objects or concepts being reasoned about. This means that logical reasoning must abstract away from all such special features and be equally valid when applied in other domains. Arguments are accepted as logical based on their conformance to a general *form* rather than because of the specific *content* they treat. For instance, compare this traditional example:

All men are mortal
Socrates is a man
Therefore Socrates is mortal

with the following reasoning drawn from mathematics:

All positive integers are the sum of four integer squares
15 is a positive integer
Therefore 15 is the sum of four integer squares

These two arguments are both correct, and both share a common pattern:

All X are Y
a is X
Therefore a is Y

This pattern of inference is logically valid, since its validity does not depend on the content: the meanings of 'positive integer', 'mortal' etc. are irrelevant. We can substitute anything we like for these X, Y and a, provided we respect grammatical categories, and the statement is still valid. By contrast, consider the following reasoning:

All Athenians are Greek
Socrates is an Athenian
Therefore Socrates is mortal

Even though the conclusion is perfectly true, this is not logically valid, because it does depend on the content of the terms involved. Other arguments with the same superficial form may well be false, e.g.

All Athenians are Greek
Socrates is an Athenian
Therefore Socrates is beardless

The first argument can, however, be turned into a logically valid one by making explicit a hidden assumption 'all Greeks are mortal'. Now the argument is an instance of the general logically valid form:

All G are M
All A are G
s is A
Therefore s is M

At first sight, this forensic analysis of reasoning may not seem very impressive. Logically valid reasoning never tells us anything fundamentally new about the world – as Wittgenstein (1922) says, 'I know nothing about the weather when I know that it is either raining or not raining'. In other words, if we *do* learn something new about the world from a chain of reasoning, it must contain a step that is *not* purely logical. Russell, quoted in Schilpp (1944) says:

Hegel, who deduced from pure logic the whole nature of the world, including the non-existence of asteroids, was only enabled to do so by his logical incompetence.†

But logical analysis can bring out clearly the necessary relationships *between* facts about the real world and show just where possibly unwarranted assumptions enter into them. For example, from 'if it has just rained, the ground is wet' it follows logically that 'if the ground is not wet, it has not just rained'. This is an instance of a general principle called *contraposition*: from 'if P then Q' it follows that 'if not Q then not P'. However, passing from 'if P then Q' to 'if Q then P' is *not* valid in general, and we see in this case that we cannot deduce 'if the ground is wet, it has just rained', because it might have become wet through a burst pipe or device for irrigation.

Such examples may be, as Locke (1689) put it, 'trifling', but elementary logical fallacies of this kind are often encountered. More substantially, deductions in mathematics are very far from trifling, but have preoccupied and often defeated some of the greatest intellects in human history. Enormously lengthy and complex chains of logical deduction can lead from simple and apparently indubitable assumptions to sophisticated and unintuitive theorems, as Hobbes memorably discovered (Aubrey 1898):

> Being in a Gentleman's Library, Euclid's Elements lay open, and 'twas the 47 *El. libri* 1 [Pythagoras's Theorem]. He read the proposition. *By G—*, sayd he (he would now and then sweare an emphaticall Oath by way of emphasis) *this is impossible!* So he reads the Demonstration of it, which referred him back to such a Proposition; which proposition he read. That referred him back to another, which he also read. *Et sic deinceps* [and so on] that at last he was demonstratively convinced of that trueth. This made him in love with Geometry.

Indeed, Euclid's seminal work *Elements of Geometry* established a particular style of reasoning that, further refined, forms the backbone of present-day mathematics. This style consists in asserting a small number of *axioms*, presumably with mathematical content, and deducing consequences from them using *purely logical reasoning*.‡ Euclid himself didn't quite achieve a complete separation of logical and non-logical, but his work was finally perfected by Hilbert (1899) and Tarski (1959), who made explicit some assumptions such as 'Pasch's axiom'.

† To be fair to Hegel, the word *logic* was often used in a broader sense until quite recently, and what we consider logic would have been called specifically *deductive logic*, as distinct from *inductive logic*, the drawing of conclusions from observed data as in the physical sciences.

‡ Arguably this approach is foreshadowed in the Socratic method, as reported by Plato. Socrates would win arguments by leading his hapless interlocutors from their views through chains of apparently inevitable consequences. When absurd consequences were derived, the initial position was rendered untenable. For this method to have its uncanny force, there must be no doubt at all over the steps, and no hidden assumptions must be sneaked in.

1.2 Calculemus!

'Reasoning is reckoning'. In the epigraph of this book we quoted Hobbes on the similarity between logical reasoning and numerical calculation. While Hobbes deserves credit for making this better known, the idea wasn't new even in 1651.† Indeed the Greek word *logos*, used by Plato and Aristotle to mean reason or logical thought, can also in other contexts mean computation or reckoning. When the works of the ancient Greek philosophers became well known in medieval Europe, *logos* was usually translated into *ratio*, the Latin word for reckoning (hence the English words rational, ratiocination, etc.). Even in current English, one sometimes hears 'I reckon that . . . ', where 'reckon' refers to some kind of reasoning rather than literally to computation.

However, the connection between reasoning and reckoning remained little more than a suggestive slogan until the work of Gottfried Wilhelm von Leibniz (1646–1716). Leibniz believed that a system for reasoning by calculation must contain two essential components:

- a universal language (*characteristica universalis*) in which anything can be expressed;
- a calculus of reasoning (*calculus ratiocinator*) for deciding the truth of assertions expressed in the *characteristica*.

Leibniz dreamed of a time when disputants unable to agree would not waste much time in futile argument, but would instead translate their disagreement into the *characteristica* and say to each other '*calculemus*' (let us calculate). He may even have entertained the idea of having a machine do the calculations. By this time various mechanical calculating devices had been designed and constructed, and Leibniz himself in 1671 designed a machine capable of multiplying, remarking:

> It is unworthy of excellent men to lose hours like slaves in the labour of calculations which could safely be relegated to anyone else if machines were used.

So Leibniz foresaw the essential components that make automated reasoning possible: a language for expressing ideas precisely, rules of calculation for manipulating ideas in the language, and the mechanization of such calculation. Leibniz's concrete accomplishments in bringing these ideas to fruition were limited, and remained little-known until recently. But though his work had limited direct influence on technical developments, his dream still resonates today.

† The Epicurian philosopher Philodemus, writing in the first century B.C., introduced the term *logisticos* (λογιστικός) to describe logic as the science of calculation.

1.3 Symbolism

Leibniz was right to draw attention to the essential first step of developing an appropriate language. But he was far too ambitious in wanting to express all aspects of human thought. Eventual progress came rather by extending the scope of the symbolic notations already used in mathematics. As an example of this notation, we would nowadays write '$x^2 \leq y + z$' rather than 'x multiplied by itself is less than or equal to the sum of y and z'. Over time, more and more of mathematics has come to be expressed in formal symbolic notation, replacing natural language renderings. Several sound reasons can be identified.

First, a well-chosen symbolic form is usually shorter, less cluttered with irrelevancies, and helps to express ideas more briefly and intuitively (at least to cognoscenti). For example Leibniz's own notation for differentiation, $\mathrm{d}y/\mathrm{d}x$, nicely captures the idea of a ratio of small differences, and makes theorems like the chain rule $\mathrm{d}y/\mathrm{d}x = \mathrm{d}y/\mathrm{d}u \cdot \mathrm{d}u/\mathrm{d}x$ look plausible based on the analogy with ordinary algebra.

Second, using a more stylized form of expression can avoid some of the ambiguities of everyday language, and hence communicate meaning with more precision. Doubts over the exact meanings of words are common in many areas, particularly law.[†] Mathematics is not immune from similar basic disagreements over exactly what a theorem says or what its conditions of validity are, and the consensus on such points can change over time (Lakatos 1976; Lakatos 1980).

Finally, and perhaps most importantly, a well-chosen symbolic notation can contribute to making mathematical reasoning itself easier. A simple but outstanding example is the 'positional' representation of numbers, where a number is represented by a sequence of numerals each implicitly multiplied by a certain power of a 'base'. In decimal the base is 10 and we understand the string of digits '179' to mean:

$$179 = 1 \times 10^2 + 7 \times 10^1 + 9 \times 10^0.$$

In binary (currently used by most digital computers) the base is 2 and the same number is represented by the string 10110011:

$$10110011 = 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0.$$

[†] For example 'Since the object of ss 423 and 425 of the Insolvency Act 1986 was to remedy the avoidance of debts, the word 'and' between paragraphs (a) and (b) of s 423(2) must be read conjunctively and not disjunctively.' (Case Summaries, *Independent* newspaper, 27th December 1993.)

These positional systems make it very easy to perform important operations on numbers like comparing, adding and multiplying; by contrast, the system of Roman numerals requires more involved algorithms, though there is evidence that many Romans were adept at such calculations (Maher and Makowski 2001). For example, we are normally taught in school to add decimal numbers digit-by-digit from the right, propagating a carry leftwards by adding one in the next column. Once it becomes second nature to follow the rules, we can, and often do, forget about the underlying meaning of these sequences of numerals. Similarly, we might transform an equation $x-3=5-x$ into $x=3+5-x$ and then to $2x=5+3$ without pausing each time to think about *why* these rules about moving things from one side of the equation to the other are valid. As Whitehead (1919) says, symbolism and formal rules of manipulation:

> [...] have invariably been introduced to make things easy. [...] by the aid of symbolism, we can make transitions in reasoning almost mechanically by the eye, which otherwise would call into play the higher faculties of the brain. [...] Civilisation advances by extending the number of important operations which can be performed without thinking about them.

Indeed, such formal rules can be followed reliably by people who do *not* understand the underlying justification, or by computers. After all, computers are expressly designed to follow formal rules (programs) quickly and reliably. They do so without regard to the underlying justification, and will faithfully follow even erroneous sets of rules (programs with 'bugs').

1.4 Boole's algebra of logic

The word *algebra* is derived from the Arabic 'al-jabr', and was first used in the ninth century by Mohammed al-Khwarizmi (ca. 780–850), whose name lies at the root of the word 'algorithm'. The term 'al-jabr' literally means 'reunion', but al-Khwarizmi used it to describe in particular his method of solving equations by collecting together ('reuniting') like terms, e.g. passing from $x+4=6-x$ to $2x=6-4$ and so to the solution $x=1$.[†] Over the following centuries, through the European renaissance, algebra continued to mean, essentially, rules of manipulation for solving equations.

During the nineteenth century, algebra in the traditional sense reached its limits. One of the central preoccupations had been the solving of equations of higher and higher degree, but Niels Henrik Abel (1802–1829) proved in

[†] The first use of the phrase in Europe was nothing to do with mathematics, but rather the appellation 'algebristas' for Spanish barbers, who also set ('reunited') broken bones as a sideline to their main business.

1824 that there is no general way of solving polynomial equations of degree 5 and above using the 'radical' expressions that had worked for lower degrees. Yet at the same time the scope of algebra expanded and it became generalized. Traditionally, variables had stood for real numbers, usually unknown numbers to be determined. However, it soon became standard practice to apply all the usual rules of algebraic manipulation to the 'imaginary' quantity i assuming the formal property $i^2 = -1$. Though this procedure went for a long time without any rigorous justification, it was effective.

Algebraic methods were even applied to objects that were not numbers in the usual sense, such as matrices and Hamilton's 'quaternions', even at the cost of abandoning the usual 'commutative law' of multiplication $xy = yx$. Gradually, it was understood that the underlying interpretation of the symbols could be ignored, provided it was established once and for all that the rules of manipulation used are all valid under that interpretation. The state of affairs was described clear-sightedly by George Boole (1815–1864).

> They who are acquainted with the present state of the theory of Symbolic Algebra, are aware, that the validity of the processes of analysis does not depend upon the interpretation of the symbols which are employed, but solely on their laws of combination. Every system of interpretation which does not affect the truth of the relations supposed, is equally admissible, and it is true that the same process may, under one scheme of interpretation, represent the solution of a question on the properties of numbers, under another, that of a geometrical problem, and under a third, that of a problem of dynamics or optics. (Boole 1847)

Boole went on to observe that nevertheless, by historical or cultural accident, all algebra at the time involved objects that were in some sense quantitative. He introduced instead an algebra whose objects were to be interpreted as 'truth-values' of true or false, and where variables represent *propositions*.[†] By a proposition, we mean an assertion that makes a declaration of fact and so may meaningfully be considered either true or false. For example, '$1 < 2$', 'all men are mortal', 'the moon is made of cheese' and 'there are infinitely many prime numbers p such that $p + 2$ is also prime' are all propositions, and according to our present state of knowledge, the first two are true, the third false and the truth-value of the fourth is unknown (this is the 'twin primes conjecture', a famous open problem in mathematics).

We are familiar with applying to numbers various arithmetic operations like unary 'minus' (negation) and binary 'times' (multiplication) and 'plus' (addition). In an exactly analogous way, we can combine truth-values using

[†] Actually Boole gave two different but related interpretations: an 'algebra of classes' and an 'algebra of propositions'; we'll focus on the latter.

so-called *logical connectives*, such as unary 'not' (logical negation or complement) and binary 'and' (conjunction) and 'or' (disjunction).† And we can use letters to stand for arbitrary *propositions* instead of *numbers* when we write down expressions. Boole emphasized the connection with ordinary arithmetic in the precise formulation of his system and in the use of the familiar algebraic notation for many logical constants and connectives:

0	false
1	true
pq	p and q
$p+q$	p or q

On this interpretation, many of the familiar algebraic laws still hold. For example, 'p and q' always has the same truth-value as 'q and p', so we can assume the commutative law $pq = qp$. Similarly, since 0 is false, '0 and p' is false whatever p may be, i.e. $0p = 0$. But the Boolean algebra of propositions satisfies additional laws that have no counterpart in arithmetic, notably the law $p^2 = p$, where p^2 abbreviates pp.

In everyday English, the word 'or' is ambiguous. The complex proposition 'p or q' may be interpreted either inclusively (p or q or both) or exclusively (p or q but not both).‡ In everyday usage it is often implicit that the two cases are mutually exclusive (e.g. 'I'll do it tomorrow or the day after'). Boole's original system restricted the algebra so that $p+q$ only made sense if $pq = 0$, rather as in ordinary algebra x/y only makes sense if $y \neq 0$. However, following Boole's successor William Stanley Jevons (1835–1882), it became customary to allow use of 'or' without restriction, and interpret it in the inclusive sense. We will always understand 'or' in this now-standard sense, 'p or q' meaning 'p or q *or both*'.

Mechanization

Even before Boole, machines for logical deduction had been developed, notably the 'Stanhope demonstrator' invented by Charles, third Earl of Stanhope (1753–1816). Inspired by this, Jevons (1870) subsequently designed and built his 'logic machine', a piano-like device that could perform certain calculations in Boole's algebra of classes. However, the limits of mechanical

† Arguably *disjunction* is something of a misnomer, since the two truth-values need not be disjoint, so some like Quine (1950) prefer *alternation*. And the word 'connective' is a misnomer in the case of unary operations like 'not', since it does not connect two propositions, but merely negates a single one. However, both usages are well-established.

‡ Latin, on the other hand, has separate phrases 'p vel q' and 'aut p aut q' for the inclusive and exclusive readings, respectively.

engineering and the slow development of logic itself meant that the mechanization of reasoning really started to develop somewhat later, at the start of the modern computer age. We will cover more of the history later in the book in parallel with technical developments. Jevons's original machine can be seen in the Oxford Museum for the History of Science.†

Logical form

In Section 1.1 we talked about arguments 'having the same form', but did not define this precisely. Indeed, it's hard to do so for arguments expressed in English and other natural languages, which often fail to make the logical structure of sentences apparent: superficial similarities can disguise fundamental structural differences, and vice versa. For example, the English word 'is' can mean 'has the property of being' ('4 is even'), or it can mean 'is the same as' ('$2 + 2$ is 4'). This example and others like it have often generated philosophical confusion.

Once we have a precise symbolism for logical concepts (such as Boole's algebra of logic) we can simply say that two arguments have the same form if they are both instances of the same formal expression, consistently replacing variables by other propositions. And we can use the formal language to make a mathematically precise definition of logically valid arguments. This is not to imply that the definition of logical form and of purely logical argument is a philosophically trivial question; quite the contrary. But we are content not to solve this problem but to finesse it by adopting a precise mathematical definition, rather as Hertz (1894) evaded the question of what 'force' means in mechanics. After enough concrete experience we will briefly consider (Section 7.8) how our demarcation of the logical arguments corresponds to some traditional philosophical distinctions.

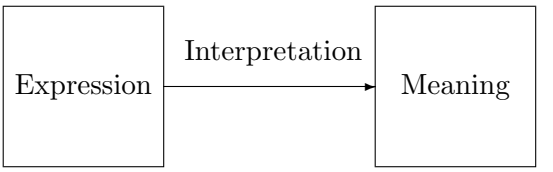
1.5 Syntax and semantics

An unusual feature of logic is the careful separation of symbolic expressions and what they stand for. This point bears emphasizing, because in everyday mathematics we often pass unconsciously to the mathematical objects denoted by the symbols. For example when we read and write '12' we think of it as a number, a member of the set $\mathbb{N}$, not as a sequence of two numeral symbols used to represent that number. However, when we want to make precise our formal manipulations, whether these be adding decimal numbers

† See `www.mhs.ox.ac.uk/database/index.htm?fname=brief&invno=18230` for some small pictures.

digit-by-digit or using algebraic laws to rearrange symbolic expressions, we need to maintain the distinction. After all, when deriving equations like $x + y = y + x$, the whole point is that the mathematical objects denoted are the same; we cannot directly talk about such manipulations if we only consider the underlying meaning.

Typically then, we are concerned with (i) some particular set of allowable formal expressions, and (ii) their corresponding meanings. The two are sharply distinguished, but are connected by an *interpretation*, which maps expressions to their meanings:



The distinction between formal expressions and their meanings is also important in linguistics, and we'll take over some of the jargon from that subject. Two traditional subfields of linguistics are *syntax*, which is concerned with the grammatical formation of sentences, and *semantics*, which is concerned with their meanings. Similarly in logic we often refer to methods as 'syntactic' if 'like algebraic manipulations' they are considered in isolation from meanings, and 'semantic' or 'semantical' if meanings play an important role. The words 'syntax' and 'semantics' are also used in linguistics with more concrete meanings, and these too are adopted in logic.

- The *syntax* of a language is a system of grammar laying out rules about how to produce or recognize grammatical phrases and sentences. For example, we might consider 'I went to the shop' grammatical English but not 'I shop to the went' because the noun and verb are swapped. In logical systems too, we will often have rules telling us how to generate or recognize well-formed expressions, perhaps for example allowing '$x + 1$' but not '$+1\times$'.
- The *semantics* of a particular word, symbol, sign or phrase is simply its meaning. More broadly, the semantics of a language is a systematic way of ascribing such meanings to all the (grammatical) expressions in the language. Translated into linguistic jargon, choosing an interpretation amounts exactly to giving a semantics to the language.

Object language and metalanguage

It may be confusing that we will be describing formal rules for performing logical reasoning, and yet will reason *about* those rules using . . . logic! In this connection, it's useful to keep in mind the distinction between the (formal) logic we are talking about and the (everyday intuitive) logic we are using to reason about it. In order to emphasize the contrast we will sometimes deploy the following linguistic jargon. A *metalanguage* is a language used to talk *about* another distinct *object language*, and likewise a *metalogic* is used to reason about an *object logic*. Thus, we often call the theorems we derive about formal logic and automated reasoning systems *metatheorems* rather than merely *theorems*. This is not (only) to sound more grandiose, but to emphasize the distinction from 'theorems' expressed *inside* those formal systems. Likewise, metalogical reasoning applied to formalized mathematical proofs is often called *metamathematics* (see Section 7.1). By the way, our chosen programming language OCaml is derived from Edinburgh ML, which was expressly designed for writing theorem proving programs (Gordon, Milner and Wadsworth 1979) and whose name stands for Meta Language. This object–meta distinction (Tarski 1936; Carnap 1937) isn't limited to logical languages. For instance, in a Russian language lesson given in English, we can consider Russian to be the object language and English the metalanguage.

Abstract and concrete syntax

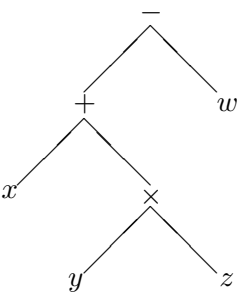
Fine details of syntax are of no fundamental importance. Some mathematics is typed, some is handwritten, and people make various essentially arbitrary choices that do not change anything about the structural way symbols are used together. When mechanizing logic on the computer, we will, for simplicity, restrict ourselves to the usual stock of ASCII characters,[†] which includes unaccented Latin letters, numbers and some common punctuation signs and spaces. For the fancy letters and special symbols that many logicians use, we will use other letters or words, e.g. 'forall' instead of '$\forall$'. We will, however, continue to employ the usual symbols in theoretical discussions. This continual translation may even be helpful to the reader who hasn't seen or understood the symbols before.

Regardless of how the symbolic expressions are read or written, it's more convenient to manipulate them in a form better reflecting their structure. Consider the expression '$x + y \times z - w$' in ordinary algebra. This linear form

[†] See `en.wikipedia.org/wiki/ASCII`.

obscures the meaningful structure. To understand which operators have been applied to which subexpressions, or even what constitutes a subexpression, we need to know rules of precedence and associativity, e.g. that '$\times$' 'binds tighter' than '$+$'. For instance, despite their apparent similarity in the linear form, '$y \times z$' is a subexpression while '$x + y$' is not. Even if we make the structure explicit by fully bracketing it as '$(x + (y \times z)) - w$', basic useful operations on expressions like finding subexpressions, or evaluating the expression for particular values of the variables, become tiresome to describe precisely; one needs to shuffle back and forth over the formula matching up brackets.

A 'tree' structure is much better: just as a family tree makes relations among family members clearly apparent, a tree representation of an expression displays its structure and makes most important manipulations straightforward. As in genealogy, it's customary to draw trees growing downwards on the printed page, so the same expression might be represented as follows:



Generally we refer to the (mainly linear) format used by people as the *concrete syntax*, and the structural (typically tree-like) form used for manipulations as the *abstract syntax*. Trees like the above are often called *abstract syntax trees* (ASTs) and are widely used as the internal representation of formal languages in all kinds of symbolic programs, including the compilers that translate high-level programming languages into machine instructions.

Despite their making the structure of an expression clearer, most people prefer not to think or communicate using trees, but to use the less structured concrete syntax.[†] Hence in our theorem-proving programs we will need to translate input from concrete syntax to abstract syntax, and translate output back from abstract syntax to concrete syntax. These two tasks, known to computer scientists as *parsing* and *prettyprinting*, are now well understood

[†] This is not to say that concrete syntax is necessarily a linear sequence of symbols. Mathematicians often use semi-graphical symbolism (matrix notation, commutative diagrams), and the pioneering logical notation introduced by Frege (1879) was tree-like.

and fairly routine. The small overhead of writing parsers and prettyprinters is amply repaid by the greater convenience of the tree form for internal manipulation. There are enthusiastic advocates of systems of concrete syntax such as 'Polish notation', 'reverse Polish notation (RPN)' and LISP 'S-expressions', where our expression would be denoted, respectively, by

```
- + x × y z w
x y z × + w -
(- (+ x (× y z)) w)
```

but we will use more traditional notation, with infix operators like '+' and rules of precedence and bracketing.[†]

1.6 Symbolic computation and OCaml

In the early days of modern computing it was commonly believed that computers were essentially devices for numeric calculation (Ceruzzi 1983). Their input and output devices were certainly biased in that direction: when Samuels wrote the first checkers (draughts) program at IBM in 1948, he had to encode the output as a number because that was all that could be printed.[‡] However, it had already been recognized, long before Turing's theoretical construction of a universal machine (see Section 7.5), that the potential applicability of computers was much wider. For example, Ada Lovelace observed in 1842 (Huskey and Huskey 1980):[§]

> Many persons who are not conversant with mathematical studies, imagine that because the business of [Babbage's analytical] engine is to give its results in numerical notation, the nature of its processes must consequently be arithmetical and numerical, rather than algebraical and analytical. This is an error. The engine can arrange and combine its numerical quantities exactly as if they were letters or any other general symbols; and in fact it might bring out its results in algebraical notation, were provisions made accordingly.

There are now many programs that perform symbolic computation, including various quite successful 'computer algebra systems' (CASs). Theorem proving programs bear a strong family resemblance to CASs, and even overlap in some of the problems they can solve (see Section 5.11, for example).

[†] Originally the spartan syntax of LISP 'S-expressions' was to be supplemented by a richer and more conventional syntax of 'M-expressions' (meta-expressions), and this is anticipated in some of the early publications like the LISP 1.5 manual (McCarthy 1962). However, such was the popularity of S-expressions that M-expressions were seldom implemented and never caught on.

[‡] Related in his speech to the 1985 International Joint Conference on Artificial Intelligence.

[§] See `www.fourmilab.to/babbage/sketch.html`.

The preoccupations of those doing symbolic computation have influenced their favoured programming languages. Whereas many system programmers favour C, numerical analysts FORTRAN and so on, symbolic programmers usually prefer higher-level languages that make typical symbolic operations more convenient, freeing the programmer from explicit details of memory representation etc. We've chosen to use Objective CAML (OCaml) as the vehicle for the programming examples in this book. Our code does not use any of OCaml's more exotic features, and should be easy to port to related functional languages such as F♯, Standard ML or Haskell.

Our insistence on using explicit OCaml code may be disquieting for those with no experience of computer programming, or for those who only know imperative and relatively low-level languages like C or Java. However, we hope that with the help of Appendix 2 and additional study of some standard texts recommended at the end of this chapter, the determined reader will pick up enough OCaml to follow the discussion and play with the code. As a gentle introduction to symbolic computation in OCaml, we will now implement some simple manipulations in ordinary algebra, a domain that will be familiar to many readers.

The first task is to define a datatype to represent the abstract syntax of algebraic expressions. We will allow expressions to be built from numeric constants like `0`, `1` and `33` and named variables like `x` and `y` using the operations of addition ('`+`') and multiplication ('`*`'). Here is the corresponding recursive datatype declaration:

```
type expression =
   Var of string
 | Const of int
 | Add of expression * expression
 | Mul of expression * expression;;
```

That is, an expression is either a variable identified by a string, a constant identified by its integer value, or an addition or multiplication operator applied to two subexpressions. (A '`*`' indicates that the domain of a type constructor is a Cartesian product, so it can take two expressions as arguments. It is nothing to do with the multiplication being defined!) We can use the syntax constructors introduced by this type definition to create the symbolic representation for any particular expression, such as $2 \times x + y$:

```
# Add(Mul(Const 2,Var "x"),Var "y");;
- : expression = Add (Mul (Const 2, Var "x"), Var "y")
```

A simple but representative example of symbolic computation is applying specified transformation rules like $0 + x \longrightarrow x$ and $3 + 5 \longrightarrow 8$ to 'simplify' an expression. Each rule is expressed in OCaml by a starting and finishing *pattern*, e.g. `Add(Const(0),x) -> x` for a transformation $0 + x \longrightarrow x$. (The special pattern '`_`' matches anything, so the last line ensures that if none of the other patterns match, `expr` is returned unchanged.) When the function is applied, OCaml will run through the rules in order and apply the first one whose starting pattern matches the input expression `expr`, replacing variables like `x` by the relevant subexpression.

```
let simplify1 expr =
  match expr with
    Add(Const(m),Const(n)) -> Const(m + n)
  | Mul(Const(m),Const(n)) -> Const(m * n)
  | Add(Const(0),x) -> x
  | Add(x,Const(0)) -> x
  | Mul(Const(0),x) -> Const(0)
  | Mul(x,Const(0)) -> Const(0)
  | Mul(Const(1),x) -> x
  | Mul(x,Const(1)) -> x
  | _ -> expr;;
```

However, simplifying just once is not necessarily adequate; we would like instead to simplify repeatedly until no further progress is possible. To do this, let us apply the above function in a bottom-up sweep through an expression tree, which will simplify in a cascaded manner. In traditional OCaml recursive style, we first simplify any immediate subexpressions as much as possible, then apply `simplify1` to the result:[†]

```
let rec simplify expr =
  match expr with
    Add(e1,e2) -> simplify1(Add(simplify e1,simplify e2))
  | Mul(e1,e2) -> simplify1(Mul(simplify e1,simplify e2))
  | _ -> simplify1 expr;;
```

Rather than a simple bottom-up sweep, a more sophisticated approach would be to mix top-down and bottom-up simplification. For example, if E is very large it would seem more efficient to simplify $0 \times E$ immediately to 0 without any examination of E. However, this needs to be implemented with care to ensure that all simplifiable subterms are simplified without the danger of looping indefinitely. Anyway, here is our simplification function in action on the expression $(0 \times x + 1) * 3 + 12$:

[†] We could leave `simplify1` out of the last line, since no simplification will be applicable to any expression reaching this case, but it seems more thematic to include it.

```
# let e = Add(Mul(Add(Mul(Const(0),Var "x"),Const(1)),Const(3)),
              Const(12));;
val e : expression =
  Add (Mul (Add (Mul (Const 0, Var "x"), Const 1), Const 3), Const 12)
# simplify e;;
- : expression = Const 15
```

Getting this far is straightforward using standard OCaml functional programming techniques: recursive datatypes to represent tree structures and the definition of functions via pattern-matching and recursion. We hope the reader who has not used similar languages before can begin to see why OCaml is appealing for symbolic computing. But of course, those who are fond of other programming languages are more than welcome to translate our code into them.

As planned, we will implement a parser and prettyprinter to translate between abstract syntax trees and concrete strings ('`x + 0`'), setting them up to be invoked automatically by OCaml for input and output of expressions. We model our concrete syntax on ordinary algebraic notation, except that in a couple of respects we will follow the example of computer languages rather than traditional mathematics. We allow arbitrarily long 'words' as variables, whereas mathematicians traditionally use mostly single letters with superscripts and subscripts; this is especially important given the limited stock of ASCII characters. And we insist that multiplication is written with an explicit infix symbol ('x `*` y'), rather than simple juxtaposition ('x y'), which later on we will use for function application. In everyday mathematics we usually rely on informal cues like variable names and background knowledge to see at once that $f(x+1)$ denotes function application whereas $y(x+1)$ denotes multiplication, but this kind of context-dependent parsing is a bit more complicated to implement.

1.7 Parsing

Translating concrete into abstract syntax is a well-understood topic because of its central importance to programming language compilers, interpreters and translators. It is now conventional to separate the transformation into two separate stages:

- lexical analysis (scanning) decomposes the sequences of input characters into 'tokens' (roughly speaking, words);
- parsing converts the linear sequences of tokens into an abstract syntax tree.

For example, lexical analysis might split the input '`v10 + v11`' into three tokens '`v10`', '`+`' and '`v11`', coalescing adjacent alphanumeric characters into words and throwing away any number of spaces (and perhaps even line breaks) between these tokens. Parsing then only has to deal with sequences of tokens and can ignore lower-level details.

Lexing

We start by classifying characters into broad groups: spaces, punctuation, symbolic, alphanumeric, etc. We treat the underscore and prime characters as alphanumeric, in deference to the usual conventions in computing ('`x_1`') and mathematics ('f''). The following OCaml predicates tell us whether a character (actually, one-character string) belongs to a certain class:†

```
let matches s = let chars = explode s in fun c -> mem c chars;;

let space = matches " \t\n\r"
and punctuation = matches "()[]{},"
and symbolic = matches "~`!@#$%^&*-+=|\\:;<>.?/"
and numeric = matches "0123456789"
and alphanumeric = matches
  "abcdefghijklmnopqrstuvwxyz_'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";;
```

A token will be either a sequence of adjacent alphanumeric characters (like '`x`' or '`size1`'), a sequence of adjacent symbolic characters ('`+`', '`<=`'), or a single punctuation character ('`(`').‡ Lexical analysis, scanning left-to-right, will assume that a token is the longest possible, for instance that '`x1`' is a single token, not two. We treat punctuation characters differently from other symbols just to avoid some counterintuitive effects of the 'longest possible token' rule, such as the detection of a token '`((`' in the string '`((x + y) + z)`'.

Next we will define an auxiliary function `lexwhile` that takes a property `prop` of characters, such as one of the classifying predicates above, and a list of input characters, separating off as a string the longest initial sequence of that list of characters satisfying `prop`:

```
let rec lexwhile prop inp =
  match inp with
    c::cs when prop c -> let tok,rest = lexwhile prop cs in c^tok,rest
  | _ -> "",inp;;
```

† Of course, this is a very inefficient procedure. However, we care even less than usual about efficiency in these routines since parsing is not usually a critical component in overall runtime.

‡ In the present example, the only meaningful symbolic tokens consist of a single character, like '`+`'. However, by allowing longer symbolic tokens we will be able to re-use this lexical analyzer unchanged in later work.

The lexical analyzer itself maps a list of input characters into a list of token strings. First any initial spaces are separated and thrown away, using `lexwhile space`. If the resulting list of characters is nonempty, we classify the first character and use `lexwhile` to separate the longest string of characters of the same class; for punctuation (or other unexpected) characters we give `lexwhile` an always-false property so it stops at once. Then we add the first character back on to the token and recursively analyze the rest of the input.

```
let rec lex inp =
  match snd(lexwhile space inp) with
    [] -> []
  | c::cs -> let prop = if alphanumeric(c) then alphanumeric
                        else if symbolic(c) then symbolic
                        else fun c -> false in
             let toktl,rest = lexwhile prop cs in
             (c^toktl)::lex rest;;
```

We can try the lexer on a typical input string, and another example reminiscent of C syntax to illustrate longer symbolic tokens.

```
# lex(explode "2*((var_1 + x') + 11)");;
- : string list =
["2"; "*"; "("; "("; "var_1"; "+"; "x'"; ")"; "+"; "11"; ")"]
# lex(explode "if (*p1-- == *p2++) then f() else g()");;
- : string list =
["if"; "("; "*"; "p1"; "--"; "=="; "*"; "p2"; "++"; ")"; "then"; "f";
 "("; ")"; "else"; "g"; "("; ")"]
```

Parsing

Now we want to transform a sequence of tokens into an abstract syntax tree. We can reflect the higher precedence of multiplication over addition by considering an expression like $2 * w + 3 * (x + y) + z$ to be a sequence of 'product expressions' (here '$2 * w$', '$3 * (x + y)$' and 'z') separated by '$+$'. In turn each product expression, say $2 * w$, is a sequence of 'atomic expressions' (here '2' and 'w') separated by '$*$'. Finally, an atomic expression is either a constant, a variable, or an arbitrary expression enclosed in brackets; note that we require *parentheses* (round brackets), though we could if we chose allow square brackets and/or braces as well. We can invent names for these three categories, say 'expression', 'product' and 'atom', and illustrate how each is built up from the others by a series of rules often called a 'BNF[†]

[†] BNF stands for 'Backus–Naur form', honouring two computer scientists who used this technique to describe the syntax of the programming language ALGOL. Similar grammars are used in formal language theory.

grammar'; read '$\longrightarrow$' as 'may be of the form' and '|' as 'or'.

$$\begin{array}{rcl} \text{expression} & \longrightarrow & \text{product} + \cdots + \text{product} \\ \text{product} & \longrightarrow & \text{atom} * \cdots * \text{atom} \\ \text{atom} & \longrightarrow & (\text{expression}) \\ & | & \text{constant} \\ & | & \text{variable} \end{array}$$

Since the grammar is already recursive ('expression' is defined in terms of itself, via the intermediate categories), we might as well use recursion to replace the repetitions:

$$\begin{array}{rcl} \text{expression} & \longrightarrow & \text{product} \\ & | & \text{product} + \text{expression} \\ \text{product} & \longrightarrow & \text{atom} \\ & | & \text{atom} * \text{product} \\ \text{atom} & \longrightarrow & (\text{expression}) \\ & | & \text{constant} \\ & | & \text{variable} \end{array}$$

This gives rise to a very direct way of parsing the input using three mutually recursive functions for the three different categories of expression, an approach known as *recursive descent parsing*. Each parsing function is given a list of tokens and returns a pair consisting of the parsed expression tree together with any unparsed input. Note that the pattern of recursion exactly matches the above grammar and simply examines tokens when necessary to decide which of several alternatives to take. For example, to parse an expression, we first parse a product, and then test whether the first unparsed character is '+'; if it is, then we make a recursive call to parse the rest and compose the results accordingly.

```
let rec parse_expression i =
  match parse_product i with
    e1,"+"::i1 -> let e2,i2 = parse_expression i1 in Add(e1,e2),i2
  | e1,i1 -> e1,i1
```

A product works similarly in terms of a parser for atoms:

```
and parse_product i =
  match parse_atom i with
    e1,"*"::i1 -> let e2,i2 = parse_product i1 in Mul(e1,e2),i2
  | e1,i1 -> e1,i1
```

and an atom parser handles the most basic expressions, including an arbitrary expression in brackets:

```
and parse_atom i =
  match i with
    [] -> failwith "Expected an expression at end of input"
  | "("::i1 -> (match parse_expression i1 with
                  e2,")"::i2 -> e2,i2
                | _ -> failwith "Expected closing bracket")
  | tok::i1 -> if forall numeric (explode tok)
               then Const(int_of_string tok),i1
               else Var(tok),i1;;
```

The 'right-recursive' formulation of the grammar means that we interpret repeated operations that lack disambiguating brackets as right-associative, e.g. $x+y+z$ as $x+(y+z)$. Had we instead defined a 'left-recursive' grammar:

$$\begin{array}{rcl} \text{expression} & \longrightarrow & \text{product} \\ & | & \text{expression} + \text{product} \end{array}$$

then $x+y+z$ would have been interpreted as $(x+y)+z$. For an associative operation like '+' it doesn't matter that much, since at least the meanings are the same, but for '−' this latter policy is clearly more appropriate.†

Finally, we define the overall parser via a wrapper function that explodes the input string, lexically analyzes it, parses the sequence of tokens and then finally checks that no input remains unparsed. We define a generic function for this, applicable to any core parser `pfn`, since it will be useful again later:

```
let make_parser pfn s =
  let expr,rest = pfn (lex(explode s)) in
  if rest = [] then expr else failwith "Unparsed input";;
```

We call our parser `default_parser`, and test it on a simple example:

```
# let default_parser = make_parser parse_expression;;
val default_parser : string -> expression = <fun>
# default_parser "x + 1";;
- : expression = Add (Var "x", Const 1)
```

But we don't even need to invoke the parser explicitly. Our setup exploits OCaml's quotation facility so that any French-style ≪quotation≫ will automatically have its body passed as a string to the function `default_parser`:‡

† Translating such a left-recursive grammar naively into recursive parsing functions would cause an infinite loop since `parse_expression` would just call itself directly right at the beginning and never get started on useful work. However, a small modification copes with this difficulty – see the definition of `parse_left_infix` in Appendix 3.

‡ OCaml's treatment of quotations is programmable; our action of feeding the string to `default_parser` is set up in the file `Quotexpander.ml`.

```
# <<(x1 + x2 + x3) * (1 + 2 + 3 * x + y)>>;;
- : expression =
Mul (Add (Var "x1", Add (Var "x2", Var "x3")),
 Add (Const 1, Add (Const 2, Add (Mul (Const 3, Var "x"), Var "y"))))
```

The process by which parsing functions were constructed from the grammar is almost mechanical, and indeed there are tools to produce parsers automatically from slightly augmented grammars. However, we thought it worthwhile to be explicit about this programming task, which is not really so difficult and provides a good example of programming with recursive functions.

1.8 Prettyprinting

For presentation to the user we need the reverse transformation, from abstract to concrete syntax. A crude but adequate solution is the following:

```
let rec string_of_exp e =
  match e with
    Var s -> s
  | Const n -> string_of_int n
  | Add(e1,e2) -> "("^(string_of_exp e1)^" + "^(string_of_exp e2)^")"
  | Mul(e1,e2) -> "("^(string_of_exp e1)^" * "^(string_of_exp e2)^")";;
```

Brackets are necessary in general to reflect the groupings in the abstract syntax, otherwise we could mistakenly print, say '$6\times(x+y)$' as '$6\times x+y$'. Our function puts brackets uniformly round each instance of a binary operator, which is perfectly correct but sometimes looks cumbersome to a human:

```
# string_of_exp <<x + 3 * y>>;;
- : string = "(x + (3 * y))"
```

We would (probably) prefer to omit the outermost brackets, and others that are implicit in rules for precedence or associativity. So let's give `string_of_exp` an additional argument for the 'precedence level' of the operator of which the expression is an immediate subexpression. Now, brackets are only needed if the current expression has a top-level operator with lower precedence than this 'outer precedence' argument.

We arbitrarily allocate precedence 2 to addition, 4 to multiplication, and use 0 at the outermost level. Moreover, we treat the operators asymmetrically to reflect right-associativity, so the left-hand recursive subcall is given a slightly higher outer precedence to force brackets if iterated instances of the same operation are left-associated.

```
let rec string_of_exp pr e =
  match e with
    Var s -> s
  | Const n -> string_of_int n
  | Add(e1,e2) ->
        let s = (string_of_exp 3 e1)^" + "^(string_of_exp 2 e2) in
        if 2 < pr then "("^s^")" else s
  | Mul(e1,e2) ->
        let s = (string_of_exp 5 e1)^" * "^(string_of_exp 4 e2) in
        if 4 < pr then "("^s^")" else s;;
```

Our overall printing function will print with starting precedence level 0 and surround the result with the kind of quotation marks we use for input:

```
let print_exp e = Format.print_string ("<<"^string_of_exp 0 e^">>");;
```

As with the parser, we can set up the printer to be invoked automatically on any result of the appropriate type, using the following magic incantation (the hash is part of the directive that is entered, not the OCaml prompt):

```
#install_printer print_exp;;
```

Now we get output quite close to the concrete syntax we would naturally type in:

```
# <<x + 3 * y>>;;
- : expression = <<x + 3 * y>>
# <<(x + 3) * y>>;;
- : expression = <<(x + 3) * y>>
# <<1 + 2 + 3>>;;
- : expression = <<1 + 2 + 3>>
# <<((1 + 2) + 3) + 4>>;;
- : expression = <<((1 + 2) + 3) + 4>>
```

The main rough edge remaining is that expressions too large to fit on one line are not split up in an intelligent way to reflect the structure via the line breaks, as in the following example. The printers we use later (see Appendix 3) make a somewhat better job of this by employing a special OCaml library `Format`.

```
# <<(x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10) *
    (y1 + y2 + y3 + y4 + y5 + y6 + y7 + y8 + y9 + y10)>>;;
- : expression =
<<(x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10) * (y1 + y2 + y3 +
y4 + y5 + y6 + y7 + y8 + y9 + y10)>>
```

Having demonstrated the basic programming needed to support symbolic computation, we will end this chapter and move on to the serious study of logic and automated reasoning.

Further reading

We confine ourselves here to general references and those for topics that we won't cover ourselves in more depth later. More specific and technical references will be presented at the end of each later chapter.

Davis (2000) and Devlin (1997) are general accounts of the development of logic and its mechanization, as well as related topics in computer science and linguistics. There are many elementary textbooks on logic such as Hodges (1977), Mates (1972) and Tarski (1941). Two logic books that, like this one, are accompanied by computer programs are Keisler (1996) and Barwise and Etchemendy (1991). There are also several books discussing carefully the role of logical reasoning in mathematics, e.g. Garnier and Taylor (1996).

Bocheński (1961), Dumitriu (1977) and Kneale and Kneale (1962) are detailed and scholarly accounts of the history of logic. Kneebone (1963) is a survey of mathematical logic which also contains a lot of historical information, while Marciszewski and Murawski (1995) shares our emphasis on mechanization. For a readable account of Jevons's logical piano and other early 'reasoning machines', starting with the Spanish mystic Ramon Lull in the thirteenth century, see Gardner (1958). MacKenzie (2001) is a historical overview of the development of automated theorem proving and its applications.

There are numerous introductions to philosophical logic that discuss issues like the notion of logical consequence in more depth; e.g. Engel (1991), Grayling (1990) and Haack (1978). Philosophically inclined readers may enjoy considering the claims of Mill (1865) and Mauthner (1901) that logical consequence is merely a psychological accident, and the polemical replies by Frege (1879) and Husserl (1900).

For further OCaml and functional programming references, see Appendix 2. The basic parsing techniques we have described are explained in detail in virtually every book ever written on compiler technology. The 'dragon book' by Aho, Sethi and Ullman (1986) has long been considered a classic, though its treatment of parsing is probably too extensive for those whose primary interest is elsewhere. A detailed theoretical analysis of what kind of parsing tasks are and aren't decidable leads naturally into the theory of computation. Davis, Sigal and Weyuker (1994) not only covers this material thoroughly, but is also a textbook on logic. For more on prettyprinting, see Oppen (1980b) and Hughes (1995).

Other discussions of theorem proving in the same implementation-oriented style as ours are given by Huet (1986), Newborn (2001) and Paulson (1992), while Gordon (1988) also describes, in similar style, the use of theorem provers within a program verification environment. Other general textbooks

on automated theorem proving are Chang and Lee (1973), Duffy (1991) and Fitting (1990), as well as some more specialized texts we will mention later.

Exercises

1.1 Modify the parser and printer to support a concrete syntax where juxtaposition is an acceptable (or the only) way of denoting multiplication.

1.2 Add an infix exponentiation operation '`^`' to the parser, printer and simplification functions. You can make it right-associative so that '`x^y^z`' is interpreted as '`x^(y^z)`'.

1.3 Add a subtraction operation to the parser, printer and simplification functions. Be careful to make subtraction associate to the left, so that $x - y - z$ is understood as $(x - y) - z$ not $x - (y - z)$. If you get stuck, you can see how similar things are done in Appendix 3.

1.4 After adding subtraction as in the previous exercise, add a unary negation operator using the same '$-$' symbol. Take care that you can parse an expression such as $x - - - x$, correctly distinguishing instances of subtraction and negation, and simplify it to 0.

1.5 Write a simplifier that uses a more intelligent traversal strategy to avoid wasteful evaluation of subterms such as E in $0 \cdot E$ or $E - E$. Write a function to generate huge expressions in order to test how much more efficient it is.

1.6 Write a more sophisticated simplifier that will put terms in a canonical polynomial form, e.g. transform $(x+1)^3-3\cdot(x+1)^2+3\cdot(2\cdot x-x)$ into x^3-2. We will eventually develop similar functions in Chapter 5.

1.7 Many concrete strings with slightly different bracketing or spacing correspond to the same abstract syntax tree, so we can't expect $\mathrm{print}(\mathrm{parse}(s)) = s$ in general. But how about $\mathrm{parse}(\mathrm{print}(e)) = e$? If not, how could you change the code to make sure it does hold? (There is a probably apocryphal story of testing an English/Russian translation program by translating the English expression 'the spirit is willing, but the flesh is weak' into Russian and back to English, resulting in 'the vodka is good and the meat is tender'. Another version has 'out of sight, out of mind' returned as 'invisible idiot'.)

2

Propositional logic

We study propositional logic in detail, defining its formal syntax in OCaml together with parsing and printing support. We discuss some of the key propositional algorithms and prove the compactness theorem, as well as indicating the surprisingly rich applications of propositional theorem proving.

2.1 The syntax of propositional logic

Propositional logic is a modern version of Boole's algebra of propositions as presented in Section 1.4.[†] It involves expressions called *formulas*[‡] that are intended to represent propositions, i.e. assertions that may be considered true or false. These formulas can be built from constants 'true' and 'false' and some basic *atomic propositions* (*atoms*) using various logical connectives ('not', 'and', 'or', etc.). The atomic propositions are like variables in ordinary algebra, and we sometimes refer to them as *propositional variables* or *Boolean variables*. As the word 'atomic' suggests, we do not analyze their internal structure; that will be considered when we treat first-order logic in the next chapter.

Representation in OCaml

We represent propositional formulas using an OCaml datatype by analogy with the type of expressions in Section 1.6. We allow the 'constant' propositions `False` and `True` and atomic formulas `Atom p`, and can build up formulas from them using the unary operator `Not` and the binary connectives

[†] Indeed, propositional logic is sometimes called 'Boolean algebra'. But this is apt to be confusing because mathematicians refer to any algebraic structure satisfying certain axioms, roughly the usual laws of algebra together with $x^2 = x$, as *a* Boolean algebra (Halmos 1963).

[‡] When consulting the literature, the reader may find the phrase *well-formed formula* (wff for short) used instead of just 'formula'. This is to emphasize that in the concrete syntax, we are only interested in strings with a syntactically valid form, not arbitrary strings of symbols.

`And`, `Or`, `Imp` ('implies') and `Iff` ('if and only if'). We defer a discussion of the exact meanings of these connectives, and deal first with immediate practicalities.

The underlying set of atomic propositions is largely arbitrary, although for some purposes it's important that it be infinite, to avoid a limit on the complexity of formulas we can consider. In abstract treatments it's common just to index the primitive propositions by number. We make the underlying type `'a` of atomic propositions a parameter of the definition of the type of formulas, so that many basic functions work equally well whatever it may be. This apparently specious generality will be useful to avoid repeated work later when we consider the extension to first-order logic. For the same reason we include two additional formula type constructors `Forall` and `Exists`. These will largely be ignored in the present chapter but their role will become clear later on.

```
type ('a)formula = False
                 | True
                 | Atom of 'a
                 | Not of ('a)formula
                 | And of ('a)formula * ('a)formula
                 | Or of ('a)formula * ('a)formula
                 | Imp of ('a)formula * ('a)formula
                 | Iff of ('a)formula * ('a)formula
                 | Forall of string * ('a)formula
                 | Exists of string * ('a)formula;;
```

Concrete syntax

As we've seen, Boole used traditional algebraic signs like '+' for the logical connectives. This makes many logical truths look beguilingly familiar, e.g.

$$p(q + r) = pq + pr$$

But some logical truths then look quite alien, such as the following, resulting from systematically exchanging 'and' and 'or' in the first formula:

$$p + qr = (p + q)(p + r)$$

In its logical guise this says that if either p holds or both q and r hold, then either p or q holds, and also either p or r holds, and vice versa. A little thought should convince the reader that this is indeed always the case; recall that 'p or q' is inclusive, meaning p or q or both.

To avoid confusion or misleading analogies with ordinary algebra, we will use special symbols for the connectives that are nowadays fairly standard.

In each row of the following table we give the English reading of each construct, followed by the standard symbolism we will adopt in discussions, then the ASCII approximations that we will support in our programs, the corresponding abstract syntax construct, and finally some other symbolisms in use. (This last column can be ignored for the purposes of this book, but may be useful when consulting the literature.)

English	Symbolic	ASCII	OCaml	Other symbols
false	$\bot$	`false`	`False`	0, F
true	$\top$	`true`	`True`	1, T
not p	$\neg p$	`~p`	`Not p`	$\overline{p}$, $-p$, $\sim p$
p and q	$p \wedge q$	`p /\ q`	`And(p,q)`	pq, $p\&q$, $p \cdot q$
p or q	$p \vee q$	`p \/ q`	`Or(p,q)`	$p + q$, $p \mid q$, p *or* q
p implies q	$p \Rightarrow q$	`p ==> q`	`Imp(p,q)`	$p \rightarrow q$, $p \supset q$
p iff q	$p \Leftrightarrow q$	`p <=> q`	`Iff(p,q)`	$p \leftrightarrow q$, $p \equiv q$, $p \sim q$

The symbol '$\vee$' is derived from the first letter of 'vel', the Latin word for inclusive or, $\top$ looks like the first letter of 'true', while $\bot$ and $\wedge$ are just mirror-images of $\top$ and $\vee$, reflecting a principle of *duality* to be explained in Section 2.4.[†] The sign for negation is close enough to the sign for arithmetical negation to be easy to remember. Some readers may have seen the symbols for implication and 'if and only if' in informal mathematics.

As with ordinary algebra, we establish rules of precedence for the connectives, overriding it by bracketing if necessary. The (quite standard) precedence order we adopt is indicated in the ordering of the table above, with '$\neg$' the highest and '$\Leftrightarrow$' the lowest. For example $p \Rightarrow q \wedge \neg r \vee s$ means $p \Rightarrow ((q \wedge (\neg r)) \vee s)$. Perhaps it would be more appropriate to give $\wedge$ and $\vee$ equal precedence, but only a few authors do that (Dijkstra and Scholten 1990) and we will follow the herd by giving $\wedge$ higher precedence.

All our binary connectives are parsed in a right-associated fashion, so $p \wedge q \wedge r$ means $p \wedge (q \wedge r)$, and so on. In informal practice, iterated implications of the form $p \Rightarrow q \Rightarrow r$ are often used as a shorthand for '$p \Rightarrow q$ and $q \Rightarrow r$', just as $x \leq y \leq z$ is for '$x \leq y$ and $y \leq z$'. For us, however, $p \Rightarrow q \Rightarrow r$ just means $p \Rightarrow (q \Rightarrow r)$, which is not the same thing.[‡]

In informal discussions, we will not make the `Atom` constructor explicit, but will try to use variable names like p, q and r for general formulas and

[†] The symbols for 'and' and 'or' are also just more angular versions of the standard symbols for set intersection and union. This is no coincidence: $x \in S \cap T$ iff $x \in S \wedge x \in T$ and $x \in S \cup T$ iff $x \in S \vee x \in T$.

[‡] It is logically equivalent to $p \wedge q \Rightarrow r$, as the reader will be able to confirm when we have defined the term precisely.

x, y and z for general atoms. For example, when we talk about a formula $x \Leftrightarrow p$, we usually mean a formula of the form `Iff(Atom(x),p)`.

Generic parsing and printing

We set up automated parsing and printing support for formulas, just as we did for ordinary algebraic expressions in Sections 1.7–1.8. Since the details are not important for present purposes, a detailed description of the code is deferred to Appendix 3. We do want to emphasize, however, that since the type of formulas is parametrized by a type of atomic propositions, the parsing and printing functions are similarly parametrized. The function `parse_formula` has type:

```
# parse_formula;;
- : (string list -> string list -> 'a formula * string list) *
    (string list -> string list -> 'a formula * string list) ->
    string list -> string list -> 'a formula * string list
= <fun>
```

This takes as additional arguments a pair of parsers for atoms and a list of strings. For present purposes the first atom parser in the pair and the list of strings can essentially be ignored; they will be used when we extend parsing to first-order formulas in the next chapter, the former to handle special infix atomic formulas like $x < y$ and the latter to retain a context of non-propositional variables. Similarly, `print_qformula` (print a formula with quotation marks) has type:

```
# print_qformula;;
- : (int -> 'a -> unit) -> 'a formula -> unit = <fun>
```

expecting a basic 'primitive proposition printer' (which as well as the proposition gets supplied with the current precedence level) and producing a printer for the overall type of formulas.

Primitive propositions

Although many functions will be generic, it makes experimentation with some of the operations easier if we fix on a definite type of primitive propositions. Accordingly we define the following type of primitive propositions indexed by names (i.e. strings):

```
type prop = P of string;;
```

We define the following to get the name of a proposition:

```
let pname(P s) = s;;
```

Now we just need to provide a parser for atomic propositions, which is quite straightforward. For reasons explained in Appendix 3 we need to check that the first input character is not a left bracket, but otherwise we just take the first token in the input stream as the name of a primitive proposition:

```
let parse_propvar vs inp =
  match inp with
    p::oinp when p <> "(" -> Atom(P(p)),oinp
  | _ -> failwith "parse_propvar";;
```

Now we feed this to the generic formula parser, with an always-failing function for the presently unused infix atom parser and an empty list for the context of non-propositional variables:

```
let parse_prop_formula = make_parser
  (parse_formula ((fun _ _ -> failwith ""),parse_propvar) []);;
```

and we can set it to automatically apply to anything typed in quotations by:

```
let default_parser = parse_prop_formula;;
```

Now we turn to printing, constructing a (trivial) function to print propositional variables, ignoring the additional precedence argument:

```
let print_propvar prec p = print_string(pname p);;
```

and then setting up and installing the overall printer:

```
let print_prop_formula = print_qformula print_propvar;;

#install_printer print_prop_formula;;
```

We are now in an environment where propositional formulas will be automatically parsed and printed, e.g.:

```
# <<p \/ q ==> r>>;;
- : prop formula = <<p \/ q ==> r>>
# let fm = <<p ==> q <=> r /\ s \/ (t <=> ~ ~u /\ v)>>;;
val fm : prop formula = <<p ==> q <=> r /\ s \/ (t <=> ~(~u) /\ v)>>
```

(Note that the space between the two negation symbols is necessary or it would be interpreted as a single token, resulting in a parse error.)

The printer is designed to split large formulas across lines in a reasonable fashion:

```
# And(fm,fm);;
- : prop formula =
<<(p ==> q <=> r /\ s \/ (t <=> ~(~u) /\ v)) /\
  (p ==> q <=> r /\ s \/ (t <=> ~(~u) /\ v))>>
# And(Or(fm,fm),fm);;
- : prop formula =
<<((p ==> q <=> r /\ s \/ (t <=> ~(~u) /\ v)) \/
   (p ==> q <=> r /\ s \/ (t <=> ~(~u) /\ v))) /\
  (p ==> q <=> r /\ s \/ (t <=> ~(~u) /\ v))>>
```

Syntax operations

It's convenient to have syntax operations corresponding to the formula constructors usable as ordinary OCaml functions:

```
let mk_and p q = And(p,q) and mk_or p q = Or(p,q)
and mk_imp p q = Imp(p,q) and mk_iff p q = Iff(p,q)
and mk_forall x p = Forall(x,p) and mk_exists x p = Exists(x,p);;
```

Dually, it's often convenient to be able to break formulas apart without explicit pattern-matching. This function breaks apart an *equivalence* (or *bi-implication* or *biconditional*), i.e. a formula of the form $p \Leftrightarrow q$, into the pair (p, q):

```
let dest_iff fm =
  match fm with Iff(p,q) -> (p,q) | _ -> failwith "dest_iff";;
```

Similarly this function breaks apart a formula $p \wedge q$, called a *conjunction*, into its two *conjuncts* p and q:

```
let dest_and fm =
  match fm with And(p,q) -> (p,q) | _ -> failwith "dest_and";;
```

while the following recursively breaks down a conjunction into a list of conjuncts:

```
let rec conjuncts fm =
  match fm with And(p,q) -> conjuncts p @ conjuncts q | _ -> [fm];;
```

The following similar functions break down a formula $p \vee q$, called a *disjunction*, into its *disjuncts* p and q, one at the top level, one recursively:

```
let dest_or fm =
  match fm with Or(p,q) -> (p,q) | _ -> failwith "dest_or";;

let rec disjuncts fm =
  match fm with Or(p,q) -> disjuncts p @ disjuncts q | _ -> [fm];;
```

This is a top-level destructor for implications:

```
let dest_imp fm =
  match fm with Imp(p,q) -> (p,q) | _ -> failwith "dest_imp";;
```

The formulas p and q in an implication $p \Rightarrow q$ are referred to as its *antecedent* and *consequent* respectively, and we define corresponding functions:

```
let antecedent fm = fst(dest_imp fm);;
let consequent fm = snd(dest_imp fm);;
```

We'll often want to define functions by recursion over formulas, just as we did with simplification in Section 1.6. Two patterns of recursion seem sufficiently common that it makes sense to define generic functions. The following applies a function to all the atoms in a formula, but otherwise leaves the structure unchanged. It can be used, for example, to perform systematic replacement of one particular atomic proposition by another formula:

```
let rec onatoms f fm =
  match fm with
    Atom a -> f a
  | Not(p) -> Not(onatoms f p)
  | And(p,q) -> And(onatoms f p,onatoms f q)
  | Or(p,q) -> Or(onatoms f p,onatoms f q)
  | Imp(p,q) -> Imp(onatoms f p,onatoms f q)
  | Iff(p,q) -> Iff(onatoms f p,onatoms f q)
  | Forall(x,p) -> Forall(x,onatoms f p)
  | Exists(x,p) -> Exists(x,onatoms f p)
  | _ -> fm;;
```

The following is an analogue of the list iterator `itlist` for formulas, iterating a binary function over all the atoms of a formula.

```
let rec overatoms f fm b =
  match fm with
    Atom(a) -> f a b
  | Not(p) -> overatoms f p b
  | And(p,q) | Or(p,q) | Imp(p,q) | Iff(p,q) ->
        overatoms f p (overatoms f q b)
  | Forall(x,p) | Exists(x,p) -> overatoms f p b
  | _ -> b;;
```

A particularly common application is to collect together some set of attributes associated with the atoms; in the simplest case just returning the set of all atoms. We can do this by iterating a function f together with an 'append' over all the atoms, and finally converting the result to a set to remove duplicates. (We could use `union` to remove duplicates as we proceed, but the present implementation can be more efficient where the sets involved are large.)

```
let atom_union f fm = setify (overatoms (fun h t -> f(h)@t) fm []);;
```

We will soon see some illustrations of how these very general functions can be used in practice.

2.2 The semantics of propositional logic

Since propositional formulas are intended to represent assertions that may be true or false, the ultimate meaning of a formula is just one of the two *truth-values* 'true' and 'false'. However, just as an algebraic expression like $x + y + 1$ only has a definite meaning when we know what the variables x and y stand for, the meaning of a propositional formula depends on the truth-values assigned to its atomic formulas. This assignment is encoded in a *valuation*, which is a function from the set of atoms to the set of truth-values $\{\text{false}, \text{true}\}$. Given a formula p and a valuation v we then evaluate the overall truth-value by the following recursively defined function:

```
let rec eval fm v =
  match fm with
    False -> false
  | True -> true
  | Atom(x) -> v(x)
  | Not(p) -> not(eval p v)
  | And(p,q) -> (eval p v) & (eval q v)
  | Or(p,q) -> (eval p v) or (eval q v)
  | Imp(p,q) -> not(eval p v) or (eval q v)
  | Iff(p,q) -> (eval p v) = (eval q v);;
```

This is our mathematical *definition* of the semantics of propositional logic,† intended to be a natural formalization of our intuitions. (The semantics of implication is unobvious, and we discuss this at length below.) Each logical connective is interpreted by a corresponding operator on OCaml's inbuilt type `bool`. To be quite explicit about what these operators mean, we

† We may choose to regard the partially evaluated `eval p`, a function from valuations to values, as the semantics of the formula `p`, rather than make the valuation an additional argument. This is mainly a question of terminology.

can enumerate all possible combinations of inputs and see the corresponding output, for example for the `&` operator:

```
# false & false;;
- : bool = false
# false & true;;
- : bool = false
# true & false;;
- : bool = false
# true & true;;
- : bool = true
```

We can lay out this information in a *truth-table* showing how the truth-value assigned to a formula is determined by those of its immediate subformulas:†

p	q	$p \wedge q$	$p \vee q$	$p \Rightarrow q$	$p \Leftrightarrow q$
false	false	false	false	true	true
false	true	false	true	true	false
true	false	false	true	false	false
true	true	true	true	true	true

Of course, for the sake of completeness we should also include a truth-table for the unary negation:

p	$\neg p$
false	true
true	false

Let's try evaluating a formula $p \wedge q \Rightarrow q \wedge r$ in a valuation where p, q and r are set to 'true', 'false' and 'true' respectively. (We don't bother to define the value on atoms not involved in the formula, and OCaml issues a warning that we have not done so.)

```
# eval <<p /\ q ==> q /\ r>>
       (function P"p" -> true | P"q" -> false | P"r" -> true);;
...
- : bool = true
```

In another valuation, however, the formula evaluates to 'false'; readers may find it instructive to check these results by hand:

```
eval <<p /\ q ==> q /\ r>>
     (function P"p" -> true | P"q" -> true | P"r" -> false);;
```

† Truth-tables were popularized by Post (1921) and Wittgenstein (1922), though they had been used earlier by Peirce in unpublished work.

Truth-tables mechanized

We would expect the evaluation of a formula to be independent of how the valuation assigns atoms not occurring in that formula. Let us make this precise by defining a function to extract the set of atomic propositions occurring in a formula. In abstract mathematical terms, we would define `atoms` as follows by recursion on formulas:

$$\begin{aligned}
\texttt{atoms}(\bot) &= \emptyset \\
\texttt{atoms}(\top) &= \emptyset \\
\texttt{atoms}(x) &= \{x\} \\
\texttt{atoms}(\neg p) &= \texttt{atoms}(p) \\
\texttt{atoms}(p \wedge q) &= \texttt{atoms}(p) \cup \texttt{atoms}(q) \\
\texttt{atoms}(p \vee q) &= \texttt{atoms}(p) \cup \texttt{atoms}(q) \\
\texttt{atoms}(p \Rightarrow q) &= \texttt{atoms}(p) \cup \texttt{atoms}(q) \\
\texttt{atoms}(p \Leftrightarrow q) &= \texttt{atoms}(p) \cup \texttt{atoms}(q)
\end{aligned}$$

As a simple example of proof by structural induction (see appendices 1 and 2) on formulas, will show that $\texttt{atoms}(p)$ is always finite, and hence we do not distort it by interpreting it in terms of ML lists. (Of course, we need to remember that list equality and set equality are not in general the same.)

Theorem 2.1 *For any propositional formula p, the set* $\texttt{atoms}(p)$ *is finite.*

Proof By induction on the structure of the formula.

If p is $\bot$ or $\top$, then $\texttt{atoms}(p)$ is the empty set, and if p is an atom, $\texttt{atoms}(p)$ is a singleton set. In all cases, these are finite.

If p is of the form $\neg q$, then by the induction hypothesis, $\texttt{atoms}(q)$ is finite and by definition $\texttt{atoms}(\neg q) = \texttt{atoms}(q)$.

If p is of the form $q \wedge r$, $q \vee r$, $q \Rightarrow r$ or $q \Leftrightarrow r$, then $\texttt{atoms}(p) = \texttt{atoms}(q) \cup \texttt{atoms}(r)$. By the inductive hypothesis, both $\texttt{atoms}(q)$ and $\texttt{atoms}(r)$ are finite, and the union of two finite sets is finite. □

Similarly, we can justify formally the intuitively obvious fact mentioned above.

Theorem 2.2 *For any propositional formula p, if two valuations v and v' agree on the set* $\texttt{atoms}(p)$ *(i.e. $v(x) = v'(x)$ for all x in* $\texttt{atoms}(p)$*), then* $\texttt{eval}\ p\ v = \texttt{eval}\ p\ v'$.

Proof By induction on the structure of p.

If p is of the form $\bot$ or $\top$, then it is interpreted as true or false independent of the valuation.

If p is an atom x, then `atoms`$(x) = \{x\}$ and by assumption $v(x) = v'(x)$. Hence `eval` $p\ v = v(x) = v'(x) =$ `eval` $p\ v'$.

If p is of the form $q \wedge r$, $q \vee r$, $q \Rightarrow r$ or $q \Leftrightarrow r$, then `atoms`$(p) =$ `atoms`$(q) \cup$ `atoms`(r). Since the valuations agree on the union of the two sets, they agree, a fortiori, on each of `atoms`(q) and `atoms`(r). We can therefore apply the inductive hypothesis to conclude that `eval` $q\ v =$ `eval` $q\ v'$ and that `eval` $r\ v =$ `eval` $r\ v'$. Since the evaluation of p is a function of these subevaluations, `eval` $p\ v =$ `eval` $p\ v'$. □

The definition of `atoms` above can be translated directly into an OCaml function, for example using `union` for '$\cup$' and `[x]` for '$\{x\}$'. However, we prefer to define it in terms of the existing iterator `atom_union`:

```
let atoms fm = atom_union (fun a -> [a]) fm;;
```

For example:

```
# atoms <<p /\ q \/ s ==> ~p \/ (r <=> s)>>;;
- : prop list = [P "p"; P "q"; P "r"; P "s"]
```

Because the interpretation of a propositional formula p depends only on the valuation's action on the finite (say n-element) set `atoms`(p), and it can only make two choices for each, the final truth-value is completely determined by all 2^n choices for those atoms. Hence we can naturally extend the enumeration in truth-table form from the basic operations to arbitrary formulas. To implement this in OCaml, we start by defining a function that tests whether a function `subfn` returns `true` on all possible valuations of the atoms `ats`, using an existing valuation `v` for all other atoms. The space of all valuations is explored by successively modifying `v` to consider setting each atom `p` to 'true' and 'false' and calling recursively:

```
let rec onallvaluations subfn v ats =
  match ats with
    [] -> subfn v
  | p::ps -> let v' t q = if q = p then t else v(q) in
             onallvaluations subfn (v' false) ps &
             onallvaluations subfn (v' true) ps;;
```

We can apply this to a function that draws one row of the truth table and then returns 'true'. (The return value is important, because '`&`' will only

evaluate its second argument if the first argument is `true`.) This can then be used to draw the whole truth table for a formula:

```
let print_truthtable fm =
  let ats = atoms fm in
  let width = itlist (max ** String.length ** pname) ats 5 + 1 in
  let fixw s = s^String.make(width - String.length s) ' ' in
  let truthstring p = fixw (if p then "true" else "false") in
  let mk_row v =
     let lis = map (fun x -> truthstring(v x)) ats
     and ans = truthstring(eval fm v) in
     print_string(itlist (^) lis ("| "^ans)); print_newline(); true in
  let separator = String.make (width * length ats + 9) '-' in
  print_string(itlist (fun s t -> fixw(pname s) ^ t) ats "| formula");
  print_newline(); print_string separator; print_newline();
  let _ = onallvaluations mk_row (fun x -> false) ats in
  print_string separator; print_newline();;
```

Note that we print in columns of width `width` that are wide enough to hold the names of all the atoms together with `true` and `false`, plus a final space. Then all the items in the table line up nicely. For example:

```
# print_truthtable <<p /\ q ==> q /\ r>>;;
p     q     r     | formula
---------------------------
false false false | true
false false true  | true
false true  false | true
false true  true  | true
true  false false | true
true  false true  | true
true  true  false | false
true  true  true  | true
---------------------------
- : unit = ()
```

Formal and natural language

Propositional logic gives us a formal way to express some of the complex propositions that can be stated in English or other natural languages. It can be instructive to practice the formalization (translation into formal logic) of compound propositions in English. As with translation between pairs of natural languages, one can't always expect a word-for-word correspondence. But with some awareness of the structure of an informal proposition, a quite direct formalization is often possible.

In propositional logic, apart from the rules of precedence given above, we can group propositions together using the standard mathematical technique of bracketing, distinguishing for example between '$p \wedge (q \vee r)$' and '$(p \wedge q) \vee r$'.

Brackets are used quite differently in English and most other languages (to make asides like this one). Indicating the precedence in English is a more ad hoc and awkward affair and is usually done by inserting additional punctuation and 'noise words' to bracket phrases and hence disambiguate. For example we might distinguish the above two examples as 'p, and also either q or r' and 'either both p and q, or else r'. This gets unwieldy for complicated propositions, and indeed this is part of the reason for having a formal language.

Generally speaking, constructs like 'and', 'or' and 'not' can be translated quite directly from English to the corresponding logical connectives. The connective 'not' can also be implicit in English prefixes such as 'dis-' and 'un-', so we might translate 'You are either honest and kind, or dishonest, or unkind' into '$H \wedge K \vee \neg H \vee \neg K$'. However, sometimes English phrases suggest nuances beyond the merely truth-functional. For example 'and' often indicates a causal connection ('he dropped the plate and it broke') or a temporal ordering ('she climbed into bed and turned out the light'). The word 'but' arguably has the same truth-functional interpretation as 'and', yet it expresses the idea that the component propositions connect in a surprising or unfortunate way. Similarly, 'unless' can reasonably be translated by 'or', but the consequent symmetry between 'p unless q' and 'q unless p' seems surprising.

More problematical is the relationship between the implication or *conditional* $p \Rightarrow q$ and the intended English reading 'p implies q' or 'if p then q'. An apparent dissonance on this point disturbs many newcomers to formal logic, and put at least one off the subject permanently (Waugh 1991). Indeed, debates about the meaning of implication go back over 2000 years to the Megarian-Stoic logicians (Bocheński 1961). According to Sextus Empiricus, the librarian Callimachus at Alexandria said in the second century BC that 'even the crows on the rooftops are cawing about which conditionals are true'.

First of all, let's be clear that if we adopt *any* truth-functional semantics of $p \Rightarrow q$, i.e. define the truth-value of $p \Rightarrow q$ in terms of the truth-values of p and q, then the semantics we have chosen is the only reasonable one. The most fundamental principle of implication as intuitively understood is that if p and $p \Rightarrow q$ are true, then so is q; consequently if p is true and q is false, then $p \Rightarrow q$ must be false. Moreover it is also plausible that $p \wedge q \Rightarrow p$ is always true, and only the chosen semantics makes this true whatever the truth-values of p and q.

But how do we justify giving implication a truth-functional semantics at all? In everyday life, when we say 'p implies q' or 'if p then q' we usually have

in mind a causal connection between p and q. It doesn't seem reasonable to assert 'p implies q' just because it happens not to be the case that p is true while q is false. This definition commits us to accepting 'p implies q' as true whenever q is true, regardless of whether p is true or not, let alone whether it has any relation to q. Perhaps even more surprising, we also have to accept that 'p implies q' is true whenever p is *false*, regardless of q. For example, we would have to accept 'if Paris is the capital of France then $2 + 2 = 4$' and 'if the moon is made of cheese then $2 + 2 = 5$' as both true.

However, further reflection reveals that these peculiar cases do have their parallel in everyday phrases like 'if Smith wins the election then I'll eat my hat'. In mathematician's jargon we may think of such implications as being true 'trivially', with the consequent irrelevant. Similarly, if a friend plans definitely to leave town tomorrow, it seems hard to argue that his assertion 'I will leave town tomorrow or the day after' is not *true*, merely that it is a peculiar and misleading way to express himself. Again, if James is 40 years old and 2 metres tall, a remark by his mother that 'he is tall for his age' might be accepted as literally true while provoking giggles.

One can argue, roughly as the Megarian-Stoic logician Diodorus did, that the intuitive meaning of 'if p, then q' is not simply that we *do not* have $p \wedge \neg q$, but more strongly that we *cannot* under any circumstances have $p \wedge \neg q$. Rather than 'under any circumstances', Diodorus said 'at all times', being mainly concerned with propositions denoting states of affairs in the world. In mathematical assertions, the equivalent might be 'whatever the value(s) taken by the component variables'. Indeed, in everyday speech we may tend to interpret implication in a 'universalized' sense, just as we understand equations like $\mathrm{e}^{x+y} = \mathrm{e}^x \mathrm{e}^y$ as implicitly valid for all values of the variables.† However, in formal logic we need to be much more precise about which variables are universal, and in the next chapter we will introduce *quantifiers* that allow us to say 'for all x ...' and so make the universal status of variables quite explicit. Once we have this ability, our truth-functional implication can be used to build up other notions of implication with the aid of explicit quantifiers, and by then we hope the reader's qualms will have eased somewhat in any case.

Readers who are still uncomfortable may choose to regard our *material* or *truth-functional* conditional '$p \Rightarrow q$' as something distinct from the various everyday notions. The use of the same terminology may seem unfortunate,

† Quine (1950) refers to $p \Rightarrow q$ as a *conditional* statement and always reads it as 'if p then q', reserving the reading 'p implies q' for the universal validity of that conditional. Thus, implication for Quine not only contains an implicit universal quantification but is also a meta-level statement *about* propositional formulas.

but it's often the case that superficially equivalent terminologies in everyday speech and in a precise science differ. It is unlikely, for example, that words like 'energy', 'power', 'force' and 'momentum' as used in everyday speech correspond to the formal definitions of a physicist, nor 'glass' and 'metal' to those of a chemist.

In ordinary usage and our formal definitions, 'if and only if' naturally corresponds to implication in both directions: 'p if and only if q' is the same as 'p implies q and q implies p'. We've already noted that the connective is frequently called *bi-implication*, and indeed we often prove mathematical theorems of the form 'p if and only if q' by separately proving 'if p then q' and 'if q then p', just as one might prove $x = y$ by separately proving $x \leq y$ and $y \leq x$. So if the semantics of implication is accepted, that for bi-implication should be acceptable too.

2.3 Validity, satisfiability and tautology

We say that a valuation v *satisfies* a formula p if `eval` p v = true. A formula is said to be:

- a *tautology* or *logically valid* if is satisfied by *all* valuations, or equivalently, if its truth-table value is 'true' in *all* rows;
- *satisfiable* if it is satisfied by *some* valuation(s) i.e. if its truth-table value is 'true' in *at least one* row;
- *unsatisfiable* or a *contradiction* if *no* valuation satisfies it, i.e. if its truth-table value is 'false' in all rows.

Note that a tautology is also satisfiable, and as the names suggest, a formula is unsatisfiable precisely if it is not satisfiable. Moreover, in any valuation `eval` $(\neg p)$ v is false iff `eval` p v is true, so p is a tautology if and only if $\neg p$ is unsatisfiable.

The simplest tautology is just '$\top$'; a slightly more interesting example is $p \wedge q \Rightarrow p \vee q$ ('if both p and q are true then at least one of p and q is true'), while one that many people find surprising at first sight is 'Peirce's Law' $((p \Rightarrow q) \Rightarrow p) \Rightarrow p$:

```
# print_truthtable <<((p ==> q) ==> p) ==> p>>;;
p     q     | formula
----------------------
false false | true
false true  | true
true  false | true
true  true  | true
----------------------
```

The formula $p \wedge q \Rightarrow q \wedge r$ whose truth-table we first produced in OCaml is satisfiable, since its truth table has a 'true' in the last column, but it's not a tautology because it also has one 'false'. The simplest contradiction is just '$\bot$', and another simple one is $p \wedge \neg p$ ('p is both true and false'):

```
# print_truthtable <<p /\ ~p>>;;
p     | formula
---------------
false | false
true  | false
---------------
```

Intuitively speaking, tautologies are 'always true', satisfiable formulas are 'sometimes (but possibly not always) true' and contradictions are 'always false'. Indeed, the notion of a tautology is intended to capture formally, insofar as we can in propositional logic, the idea of a logical truth that we discussed in a non-technical way in the introductory chapter. A tautology is exactly analogous to an algebraic equation like $x^2 - y^2 = (x+y)(x-y)$ that is universally true whatever the values of the constituent variables. A satisfiable formula is analogous to an equation that has at least one solution but may not be universally valid, e.g. $x^2 + 2 = 3x$. A contradiction is analogous to an unsolvable equation like $0 \cdot x = 1$.

It's useful to extend the idea of (un)satisfiability from a single formula to a set of formulas: a set Γ of formulas is said to be satisfiable if there is a valuation v that *simultaneously* satisfies them all. Note the 'simultaneously': $\{p \wedge \neg q, \neg p \wedge q\}$ is unsatisfiable even though each formula by itself is satisfiable. When the set concerned is finite, $\Gamma = \{p_1, \ldots, p_n\}$, satisfiability of Γ is equivalent to that of the single formula $p_1 \wedge \cdots \wedge p_n$, as the reader will see from the definitions. However, in our later work it will be essential to consider satisfiability of infinite sets of formulas, where it cannot so directly be reduced to satisfiability of a single formula. We also use the notation $\Gamma \models q$ to mean 'for all valuations in which all $p \in \Gamma$ are true, q is true'. Note that in the case of finite $\Gamma = \{p_1, \ldots, p_n\}$, this is equivalent to the assertion that $p_1 \wedge \cdots \wedge p_n \Rightarrow q$ is a tautology. In the case $\Gamma = \emptyset$ it's common just to write $\models p$ rather than $\emptyset \models p$, both meaning that p is a tautology.

Tautology and satisfiability checking

Although we can decide the status of formulas by examining their truth tables, it's simpler to let the computer do all the work. The following function

tests whether a formula is a tautology by checking that it evaluates to 'true' for all valuations.

```
let tautology fm =
  onallvaluations (eval fm) (fun s -> false) (atoms fm);;
```

Note that as soon as any evaluation to 'false' is encountered this will, by the way `onallvaluations` was written, terminate with 'false' at once, rather than plough on through all possible valuations.

```
# tautology <<p \/ ~p>>;;
- : bool = true
# tautology <<p \/ q ==> p>>;;
- : bool = false
# tautology <<p \/ q ==> q \/ (p <=> q)>>;;
- : bool = false
# tautology <<(p \/ q) /\ ~(p /\ q) ==> (~p <=> q)>>;;
- : bool = true
```

Using the interrelationships noticed above, we can define satisfiability and unsatisfiability in terms of tautology:

```
let unsatisfiable fm = tautology(Not fm);;

let satisfiable fm = not(unsatisfiable fm);;
```

Substitution

As with algebraic identities, we expect to be able to substitute other formulas consistently for the atomic propositions in a tautology, and still get a tautology. We can define such substitution of formulas for atoms as follows, where `subfn` is a finite partial function (see Appendix 2):

```
let psubst subfn = onatoms (fun p -> tryapplyd subfn p (Atom p));;
```

For example, using the substitution function $p \mapsto p \wedge q$, which maps p to $p \wedge q$ but is otherwise undefined, we get:

```
# psubst (P"p" |=> <<p /\ q>>) <<p /\ q /\ p /\ q>>;;
- : prop formula = <<(p /\ q) /\ q /\ (p /\ q) /\ q>>
```

We will prove that substituting in tautologies yields a tautology, via a more general result that can be proved directly by structural induction on formulas:

Theorem 2.3 *For any atomic proposition x and arbitrary formulas p and q, and any valuation v, we have*[†]

$$\texttt{eval}\ (\texttt{psubst}\ (x \mapsto q)\ p)\ v = \texttt{eval}\ p\ ((x \mapsto \texttt{eval}\ q\ v)\ v).$$

Proof By induction on the structure of p. If p is $\bot$ or $\top$ then the valuation plays no role and the equation clearly holds. If p is an atom y, we distinguish two possibilities. If $y = x$ then using the definitions of substitution and evaluation we find:

$$\begin{aligned}\texttt{eval}\ (\texttt{psubst}\ (x \mapsto q)\ x)\ v &= \texttt{eval}\ q\ v \\ &= \texttt{eval}\ x\ ((x \mapsto \texttt{eval}\ q\ v)\ v).\end{aligned}$$

If, on the other hand, $y \neq x$ then:

$$\begin{aligned}\texttt{eval}\ (\texttt{psubst}\ (x \mapsto q)\ y)\ v &= \texttt{eval}\ y\ v \\ &= \texttt{eval}\ y\ ((x \mapsto \texttt{eval}\ q\ v)\ v).\end{aligned}$$

For other kinds of formula, evaluation and substitution follow the structure of the formula so the result follows easily by the inductive hypothesis. For example, if p is of the form $\neg r$ then by definition and using the inductive hypothesis for r:

$$\begin{aligned}\texttt{eval}\ (\texttt{psubst}\ (x \mapsto q)\ (\neg r))\ v &= \texttt{eval}\ (\neg(\texttt{psubst}\ (x \mapsto q)\ r))\ v \\ &= \text{not}(\texttt{eval}\ (\texttt{psubst}\ (x \mapsto q)\ r)\ v) \\ &= \text{not}(\texttt{eval}\ r\ ((x \mapsto \texttt{eval}\ q\ v)\ v)) \\ &= \texttt{eval}\ (\neg r)\ ((x \mapsto \texttt{eval}\ q\ v)\ v).\end{aligned}$$

The binary connectives all follow the same essential pattern but with two distinct formulas r and s instead of just r. □

Corollary 2.4 *If p is a tautology, x is any atom and q any other formula, then* $\texttt{psubst}\ (x \mapsto q)\ p$ *is also a tautology.*

[†] The notation $(x \mapsto a)v$ means the function v' that maps $v'(x) = a$ and $v'(y) = v(y)$ for $y \neq x$, and $x \mapsto a$ is the function that maps x to a and is undefined elsewhere (see Appendix 1). In our OCaml implementation there are corresponding operators '`|->`' and '`|=>`' for finite partial functions; see Appendix 2.

Proof By the previous theorem we have for any valuation v:

$$\texttt{eval}\ (\texttt{psubst}\ (x \models\!\Rightarrow q)\ p)\ v = \texttt{eval}\ p\ ((x \mapsto \texttt{eval}\ q\ v)\ v)$$

But since p is a tautology it evaluates to 'true' in all valuations, including the one on the right of this equation. Hence $\texttt{eval}\ (\texttt{psubst}\ (x \models\!\Rightarrow q)\ p)\ v =$ true, and since v is arbitrary, this means the formula is a tautology. □

Note that this result only applies to substituting for atoms, not arbitrary propositions. For example, $p \wedge q \Rightarrow q \wedge p$ is a tautology, but if we substitute $p \vee q$ for $p \wedge q$ it ceases to be so. This again is just as in ordinary algebra, and the fact that our substitution function is a function from names of atoms helps to enforce such a restriction. The main results are however easily generalized to substitution for multiple atoms simultaneously. These can always be done using individual substitutions repeatedly, but one might have to use additional substitutions to change variables and avoid spurious effects of later substitutions on earlier ones. For example, we would expect to be able to simultaneously substitute x for y and y for x in $x \wedge y$ to get $y \wedge x$. Yet if we perform the substitutions sequentially we get:

$$\begin{aligned} & \texttt{psubst}\ (x \models\!\Rightarrow y)\ (\texttt{psubst}\ (y \models\!\Rightarrow x)\ (x \wedge y)) \\ = \ & \texttt{psubst}\ (x \models\!\Rightarrow y)\ (x \wedge x) \\ = \ & y \wedge y. \end{aligned}$$

However, by renaming variables appropriately using other substitutions such problems can always be avoided. For example:

$$\begin{aligned} & \texttt{psubst}\ (z \models\!\Rightarrow y)\ (\texttt{psubst}\ (y \models\!\Rightarrow x)\ (\texttt{psubst}\ (x \models\!\Rightarrow z)\ (x \wedge y))) \\ = \ & \texttt{psubst}\ (z \models\!\Rightarrow y)\ (\texttt{psubst}\ (y \models\!\Rightarrow x)\ (z \wedge y)) \\ = \ & \texttt{psubst}\ (z \models\!\Rightarrow y)\ (z \wedge x) \\ = \ & y \wedge x. \end{aligned}$$

It's useful to get a feel for propositional logic by listing some common tautologies. Some are simple and plausible such as the *law of the excluded middle* '$p \vee \neg p$' stating that every proposition is either true or false. A more surprising tautology, no doubt because of the poor accord between '$\Rightarrow$' and the intuitive notion of implication, is:

```
# tautology <<(p ==> q) \/ (q ==> p)>>;;
- : bool = true
```

If $p \Rightarrow q$ is a tautology, i.e. any valuation that satisfies p also satisfies q, we say that q is a *logical consequence* of p. If $p \Leftrightarrow q$ is a tautology, i.e.

a valuation satisfies p if and only if it satisfies q, we say that p and q are *logically equivalent*. Many important tautologies naturally take this latter form, and trivially if p is a tautology then so is $p \Leftrightarrow \top$, as the reader can confirm. In algebra, given a valid equation such as $2x = x+x$, we can replace $2x$ by $x + x$ in any other expression without changing its value. Similarly, if a valuation satisfies $p \Leftrightarrow q$, then we can substitute q for p or vice versa in another formula r (even if p is not just an atom) without affecting whether the valuation satisfies r. Since we haven't formally defined substitution for non-atoms, we imagine identifying the places to substitute using some other atom x in a 'pattern' term.

Theorem 2.5 *Given any valuation v and formulas p and q such that* `eval` $p\ v =$ `eval` $q\ v$*, for any atom x and formula r we have*

$$\texttt{eval}\ (\texttt{psubst}\ (x \models\!\!\Rightarrow p)\ r)\ v = \texttt{eval}\ (\texttt{psubst}\ (x \models\!\!\Rightarrow q)\ r)\ v.$$

Proof We have `eval` (`psubst` $(x \models\!\!\Rightarrow p)\ r)\ v =$ `eval` $r\ ((x \mapsto \texttt{eval}\ p\ v)\ v)$ and `eval` (`psubst` $(x \models\!\!\Rightarrow q)\ r)\ v =$ `eval` $r\ ((x \mapsto \texttt{eval}\ q\ v)\ v)$ by Theorem 2.3. But since by hypothesis `eval` $p\ v =$ `eval` $q\ v$ these are the same. □

Corollary 2.6 *If p and q are logically equivalent, then*

$$\texttt{eval}\ (\texttt{psubst}\ (x \models\!\!\Rightarrow p)\ r)\ v = \texttt{eval}\ (\texttt{psubst}\ (x \models\!\!\Rightarrow q)\ r)\ v.$$

In particular `psubst` $(x \models\!\!\Rightarrow p)\ r$ *is a tautology iff* `psubst` $(x \models\!\!\Rightarrow q)\ r$ *is.*

Proof Since p and q are logically equivalent, we have `eval` $p\ v =$ `eval` $q\ v$ for any valuation v, and the result follows from the previous theorem. □

Some important tautologies

Without further ado, here's a list of tautologies. Many of these correspond to ordinary algebraic laws if rewritten in the Boolean symbolism, e.g. $p \wedge \bot \Leftrightarrow \bot$ to $p \cdot 0 = 0$.

$$\begin{aligned}
\neg\top &\Leftrightarrow \bot \\
\neg\bot &\Leftrightarrow \top \\
\neg\neg p &\Leftrightarrow p \\
p \wedge \bot &\Leftrightarrow \bot \\
p \wedge \top &\Leftrightarrow p \\
p \wedge p &\Leftrightarrow p
\end{aligned}$$

$$
\begin{array}{rcl}
p \wedge \neg p & \Leftrightarrow & \bot \\
p \wedge q & \Leftrightarrow & q \wedge p \\
p \wedge (q \wedge r) & \Leftrightarrow & (p \wedge q) \wedge r \\
p \vee \bot & \Leftrightarrow & p \\
p \vee \top & \Leftrightarrow & \top \\
p \vee p & \Leftrightarrow & p \\
p \vee \neg p & \Leftrightarrow & \top \\
p \vee q & \Leftrightarrow & q \vee p \\
p \vee (q \vee r) & \Leftrightarrow & (p \vee q) \vee r \\
p \wedge (q \vee r) & \Leftrightarrow & (p \wedge q) \vee (p \wedge r) \\
p \vee (q \wedge r) & \Leftrightarrow & (p \vee q) \wedge (p \vee r) \\
\bot \Rightarrow p & \Leftrightarrow & \top \\
p \Rightarrow \top & \Leftrightarrow & \top \\
p \Rightarrow \bot & \Leftrightarrow & \neg p \\
p \Rightarrow p & \Leftrightarrow & \top \\
p \Rightarrow q & \Leftrightarrow & \neg q \Rightarrow \neg p \\
p \Rightarrow q & \Leftrightarrow & (p \Leftrightarrow p \wedge q) \\
p \Rightarrow q & \Leftrightarrow & (q \Leftrightarrow q \vee p) \\
p \Leftrightarrow q & \Leftrightarrow & q \Leftrightarrow p \\
p \Leftrightarrow (q \Leftrightarrow r) & \Leftrightarrow & (p \Leftrightarrow q) \Leftrightarrow r
\end{array}
$$

The last couple are perhaps particularly surprising, since we are not accustomed to 'equations within equations' from everyday mathematics. Effectively, they show that '$\Leftrightarrow$' is a symmetric and associative operator (like '$+$' in arithmetic), in that the order and association of iterated equivalences makes no logical difference. Some other tautologies involving equivalence are given by Dijkstra and Scholten (1990) and can be checked in OCaml; they refer to the second of these tautologies as the 'Golden Rule'.

```
# tautology <<p \/ (q <=> r) <=> (p \/ q <=> p \/ r)>>;;
- : bool = true
# tautology <<p /\ q <=> ((p <=> q) <=> p \/ q)>>;;
- : bool = true
```

Another tautology in our list corresponds to the principle of *contraposition*, the equivalence of $p \Rightarrow q$ and its *contrapositive* $\neg q \Rightarrow \neg p$, or of $p \Rightarrow \neg q$ and $q \Rightarrow \neg p$. (For example 'those who mind don't matter' and 'those who

matter don't mind' are logically equivalent.) By contrast, we can confirm that $p \Rightarrow q$ and $q \Rightarrow p$ are *not* equivalent, refuting a common fallacy:

```
# tautology <<(p ==> q) <=> (~q ==> ~p)>>;;
- : bool = true
# tautology <<(p ==> ~q) <=> (q ==> ~p)>>;;
- : bool = true
# tautology <<(p ==> q) <=> (q ==> p)>>;;
- : bool = false
```

2.4 The De Morgan laws, adequacy and duality

The following important tautologies are called *De Morgan's laws*, after Augustus De Morgan, a near-contemporary of Boole who made important contributions to the field of logic.[†]

$$\begin{aligned} \neg(p \vee q) &\Leftrightarrow \neg p \wedge \neg q \\ \neg(p \wedge q) &\Leftrightarrow \neg p \vee \neg q \end{aligned}$$

An everyday example of the first is that 'I can not speak either Finnish or Swedish' means that same as 'I can not speak Finnish and I can not speak Swedish'. An example of the second is that 'I am not a wife and mother' is the same as 'either I am not a wife or I am not a mother (or both)'. Variants of the De Morgan laws, also easily seen to be tautologies, are:

$$\begin{aligned} p \vee q &\Leftrightarrow \neg(\neg p \wedge \neg q) \\ p \wedge q &\Leftrightarrow \neg(\neg p \vee \neg q) \end{aligned}$$

These are interesting because they show how to express either connective $\wedge$ and $\vee$ in terms of the other. By virtue of the above theorems on substitution, this means for example that we can 'rewrite' any formula to a logically equivalent formula not involving '$\vee$', simply by systematically replacing each subformula of the form $q \vee r$ with $\neg(\neg q \wedge \neg r)$. There are many other options for expressing some logical connectives in terms of others. For instance, using the following equivalences, one can find an equivalent for any formula using only atomic formulas, $\wedge$ and $\neg$. In the jargon, $\{\wedge, \neg\}$ is said to be an *adequate set* of connectives.

$$\begin{aligned} \bot &\Leftrightarrow p \wedge \neg p \\ \top &\Leftrightarrow \neg(p \wedge \neg p) \\ p \vee q &\Leftrightarrow \neg(\neg p \wedge \neg q) \end{aligned}$$

[†] These were given quite explicitly by John Duns the Scot (1266-1308) in his *Universam Logicam Quaestiones*. However, De Morgan was the first to put them in algebraic form.

$$p \Rightarrow q \quad \Leftrightarrow \quad \neg(p \wedge \neg q)$$
$$p \Leftrightarrow q \quad \Leftrightarrow \quad \neg(p \wedge \neg q) \wedge \neg(\neg p \wedge q)$$

Similarly the following equivalences, which we check in OCaml, show that $\{\Rightarrow, \bot\}$ is also adequate:

```
forall tautology
 [<<true <=> false ==> false>>;
  <<~p <=> p ==> false>>;
  <<p /\ q <=> (p ==> q ==> false) ==> false>>;
  <<p \/ q <=> (p ==> false) ==> q>>;
  <<(p <=> q) <=> ((p ==> q) ==> (q ==> p) ==> false) ==> false>>];;
- : bool = true
```

Is any single connective alone enough to express all the others? For the connectives we have introduced, the answer is no. We need one of the binary connectives, otherwise we could never introduce formulas that involve, and hence depend on the valuation of, more than one variable. And in fact not even the whole set $\{\top, \wedge, \vee, \Rightarrow, \Leftrightarrow\}$, without negation or falsity, forms an adequate set, so a fortiori, neither does any one binary connective individually. To see this, note that all these binary connectives with entirely 'true' arguments yield the result 'true'. (In other words, the last row of each of their truth tables contains 'true' in the final column.) Hence any formula built up from these components must evaluate to 'true' in the valuation that maps all atoms to 'true', so negation is not representable.

However, there are $2^{2^2} = 16$ possible truth-tables for a binary truth-function (there are $2^2 = 4$ rows in the truth table and each can be given one of two truth-values) and the conventional binary connectives only cover four of them. Perhaps a connective with one of the other 12 functions for its truth-table would be adequate? As argued above, any single adequate connective must have 'false' in the last row of its truth table, so that it can express negation. By a similar argument, we can also see that the first row of its truth-table must be 'true'. This only leaves us freedom of choice for the middle two rows, for which there are four choices. Two of them are trivial in that they are just the negation of one of the arguments, and hence cannot be used to build expressions whose evaluation depends on the value of more than a single atom. However, either of the other two is adequate alone: the 'not and' operation p `NAND` $q = \neg(p \wedge q)$, or the 'not or' operation p `NOR` $q = \neg(p \vee q)$, both of whose truth tables are written out below:

p	q	p NAND q	p NOR q
false	false	true	true
false	true	true	false
true	false	true	false
true	true	false	false

For example, we can express negation by $\neg p = p$ NAND p and then get $p \wedge q = \neg(p$ NAND $q)$, and we already know that $\{\wedge, \neg\}$ is adequate; NOR works similarly. In fact, once we have an adequate set of connectives, we can find formulas whose semantics corresponds to any of the other 12 truth-functions as well, as will become clear when we discuss disjunctive normal form in Section 2.6.

The adequacy of either one of the connectives NAND and NOR is well-known to electronics designers: corresponding gates are often the basic building blocks of digital circuits (see Section 2.7). Among pure logicians it's customary to denote one or the other of these connectives by $p \mid q$ and refer to '$\mid$' as the 'Sheffer stroke' (Sheffer 1913).†

Duality

In Section 1.4 we noted the choice to be made between the 'inclusive' and 'exclusive' readings of 'or'. No doubt a pleasing symmetry between 'and' and 'inclusive or' was a strong motivation for what might seem an arbitrary choice of the inclusive reading. Suppose we have a formula involving only the connectives $\bot$, $\top$, $\wedge$ and $\vee$. By its *dual* we mean the result of systematically exchanging '$\wedge$'s and '$\vee$'s and also '$\top$'s and '$\bot$'s, thus:

```
let rec dual fm =
  match fm with
    False -> True
  | True -> False
  | Atom(p) -> fm
  | Not(p) -> Not(dual p)
  | And(p,q) -> Or(dual p,dual q)
  | Or(p,q) -> And(dual p,dual q)
  | _ -> failwith "Formula involves connectives ==> or <=>";;
```

† Nowadays people usually interpret the stroke as NAND, but Sheffer originally used his stroke for NOR, and it was used in a parsimonious presentation of propositional logic by Nicod (1917). The idea had been well known to Peirce 30 years earlier. Schönfinkel (1924) elaborated it into a 'quantifier stroke', where $\phi(x) \mid_x \psi(x)$ means $\neg\exists x.\, \phi(x) \wedge \psi(x)$, and this led on to an interest in performing the same paring-down for more general mathematical expressions, and hence to his development of combinators.

for example:

```
# dual <<p \/ ~p>>;;
- : prop formula = <<p /\ ~p>>
```

A little thought shows that `dual`(`dual`(p)) = p. The key semantic property of duality is:

Theorem 2.7 `eval` (`dual` p) v = *not*(`eval` p (*not* $\circ$ v)) *for any valuation* v.

Proof This can be proved by a formal structural induction on formulas (see Exercise 2.5), but it's perhaps easier to see using more direct reasoning based on the De Morgan laws. Let p^* be the result of negating all the atoms in a formula and replacing $\bot$ by $\neg\top$, $\top$ by $\neg\bot$. We then have `eval` p (not $\circ$ v) = `eval` p^* v. Now using the De Morgan laws we can repeatedly pull the newly introduced negations up from the atoms in p^* giving a logically equivalent form:

$$\begin{aligned} \neg p \wedge \neg q &\Leftrightarrow \neg(p \vee q) \\ \neg p \vee \neg q &\Leftrightarrow \neg(p \wedge q). \end{aligned}$$

By doing so, we exchange '$\wedge$'s and '$\vee$'s, and bubble the newly introduced negation signs upwards, until we just have one additional negation sign at the top, resulting in exactly $\neg$(`dual` p). The result follows. □

Corollary 2.8 *If* p *and* q *are logically equivalent, so are* `dual` p *and* `dual` q. *If* p *is a tautology then so is* $\neg$(`dual` p).

Proof `eval` (`dual` p) v = not(`eval` p (not $\circ$ v)) = not(`eval` q (not $\circ$ v)) = `eval` (`dual` q) v. If p is a tautology, then p and $\top$ are logically equivalent, so `dual` p and `dual` $\top = \bot$ are logically equivalent and the result follows. □

For example, since $p \wedge (q \vee r)$ and $(p \wedge q) \vee (p \wedge r)$ are equivalent, so are $p \vee (q \wedge r)$ and $(p \vee q) \wedge (p \vee r)$, and since $p \vee \neg p$ is a tautology, so is $\neg(p \wedge \neg p)$.

2.5 Simplification and negation normal form

In ordinary algebra it's common to systematically transform an expression into an equivalent standard or *normal* form. One approach involves expanding and cancelling, e.g. obtaining from $(x+y)(y-x)+y+x^2$ the normal form $y^2 + y$. By putting expressions in normal form, we can sometimes see that superficially different expressions are equivalent. Moreover, if the normal